



Trinity College Dublin

Coláiste na Tríonóide, Baile Átha Cliath

The University of Dublin

Communication-Avoiding Krylov Exponential Integration for Multi-GPU Option Pricing

In partial fulfillment of MSc in High-performance Computing
in the School of Mathematics

Name: Kevin Knights
Student ID: 25378095
Supervision: Kirk M. Soodhalter

Date of Submission: 17 August 2026
Project Code: <https://github.com/kevinknights29/caksm>



Trinity College Dublin
Coláiste na Tríonóide, Baile Átha Cliath
The University of Dublin

Declaration concerning plagiarism

I hereby declare that this thesis is my own work where appropriate citations have been given and that it has not been submitted as an exercise for a degree at this or any other university.

I have read and I understand the plagiarism provisions in the General Regulations of the University Calendar for the current year, found at <http://www.tcd.ie/calendar>.

Name: Kevin Knights

Student ID: 25378095

Signature: 
.....

Date: 17 August 2026

ACKNOWLEDGEMENTS

I would like to thank my supervisor, Professor Kirk M. Soodhalter, for his support, guidance, and thoughtful feedback throughout the master's program. His teaching encouraged me to pursue research in numerical linear algebra, particularly Krylov subspace methods. As someone with an engineering background, I initially found the subject distant; his lectures, careful questions, and generosity with his time made it increasingly accessible. Our many discussions shaped this project and helped me grow as a researcher.

I am deeply grateful to my mother, Keyla, for her unconditional support despite the distance between us. Her love and encouragement sustained me through the difficult parts of this work.

I would also like to thank Jose Refojo for his patient help with modules, libraries, and configuration on the TCD HPC clusters. His support made the two-node, four-V100 experiments on Synge possible, and I am grateful for what I learned from him about CUDA and GPU computing.

I thank Richard Morrin and Niall O'Sullivan for fostering my interest in financial mathematics and computational finance. Their practical perspective, guidance, and assignments in C, C++, and quantitative finance gave me the foundation to pursue this application.

I am grateful to my classmates Edward, Louis, Eryk, and Conor for their support, friendship, and willingness to exchange ideas. I learned a great deal from them.

I am also grateful to the professors, staff, and students of the MSc in High-Performance Computing for the discussions and practical support that helped bring this work together.

ABSTRACT

This thesis studies two communication-avoiding (CA) mechanisms inside a Krylov exponential integrator for a three-asset Black–Scholes PDE. Sparse matrix–vector products move data through the memory hierarchy, while Arnoldi orthogonalization performs global reductions. Matrix-powers tiling targets the first cost and block orthogonalization targets the second, but either reformulation can lose when its redundant arithmetic, halo work, or local kernels cost more than the communication it removes.

A dimensionless regime map organizes the study. Its vertical coordinate R_v compares an Arnoldi working set with last-level-cache capacity; its horizontal coordinate R_h compares reduction latency with useful computation between reductions. The lines $R_v = 1$ and $R_h = 1$ are treated as nominal opportunity boundaries, not universal performance crossovers. They identify communication that might be recoverable; cache geometry, access efficiency, overlap, and topology determine whether a measured implementation recovers it. The coordinates are computed before the timed runs from the problem and independently calibrated machine constants.

The numerical study predicts the conditioning of matrix-powers blocks from their spectrum and compares that prediction with the implementation’s operational condition-number guard. The line $\kappa_2(\mathbf{Z}) \leq u^{-1/2}$ triggers block narrowing near the practical breakdown scale of the normal equations. At the production dimensions, the conservative sufficient condition in the cited CholeskyQR2 theorem is inconclusive and distributed Gram formation adds another rounding path. Reliability is therefore assessed directly through Cholesky success, measured orthogonality, and agreement with an independent referee. The resulting evidence is empirical for the tested blocks, with its theoretical scope stated separately. Likewise, the Krylov termination scalar is an endpoint defect indicator, not automatically a bound on the state or option-price error.

The measurements preserve two useful negative results. On the CPU, an 11–16 $\times$ reduction-count decrease produces no corresponding wall-clock improvement at $R_h = 0.080$. Cache-blocked matrix powers also falls well below its traffic ceiling because halo work and access geometry dominate. In the distributed solver, correcting an unfair recurrence-depth control changes the algorithm-relative speedup from 0.90 $\times$ on one V100 to 1.83 $\times$ on four V100s across two nodes. This is topology-specific: four V100s provide only about 1.11 $\times$ speedup over one V100 at the same block width, about 28% parallel efficiency, and tie the best one-GPU configuration. GPU profiling further shows that tile geometry can be worth more than the CA treatment itself.

The contribution is consequently a controlled mechanism study, not a claim of a state-of-the-art option-pricing solver. The benchmark measures temporal error relative to a high-accuracy solution of the same semi-discrete system; it does not by itself establish total price or Greek accuracy after spatial truncation, boundary treatment, and interpolation.

Keywords: Krylov subspace methods, communication-avoiding algorithms, matrix exponential, option pricing, roofline model, s -step methods, performance modeling

CONTENTS

1.	Introduction	6
2.	The pricing problem and its discretization	7
3.	The Krylov exponential integrator	8
3.1.	Cost structure and the sequential dependency	10
3.2.	Communication-avoiding reformulations	10
4.	Experimental platform and methodology	11
5.	Historical Niesen–Wright time-integrator comparison	12
6.	Profile and roofline	13
7.	The OpenMP scaling study	15
7.1.	Instrument design	15
7.2.	The quarantined $m(n)$ growth	16
7.3.	The locality control	17
8.	Regime analysis	18
8.1.	The coordinates	18
8.2.	The instrument and its gates	19
8.3.	Basis conditioning and the confound control	20
8.4.	The reduction ledger and small- m safety	20
8.5.	Empirical log-sensitivity to non-normality	21
8.6.	Conditioning growth with asset dimension	23
8.7.	The computability gap	24
8.8.	The horizontal mechanism	25
8.9.	The vertical mechanism	26
8.10.	Block width	27
8.11.	Placing the real operator on the map	28
9.	Recomputing the map for a GPU	29
9.1.	A cross-architecture transfer test	29
9.2.	What transfers, and what breaks	29
9.3.	The decoupled invariant	30
9.4.	Reaching the corner, and the gate that discriminates	31
9.5.	Roofline-bound kernels and the s -step crossover	32
10.	The distributed CA exponential integrator	33
10.1.	The solver	33
10.2.	Recurrence depth, and what a fair control costs	34
10.3.	Strong scaling	34
10.4.	The communication account	36
10.5.	The vertical coordinate, measured	37
10.6.	The conditioning guard on an evolving solver basis	39
10.7.	Weak scaling and the convergence stop	40
10.8.	What the integrator adds to the map	41
11.	Future work	41
12.	Conclusion	42
	References	44
A.	Supplementary figures	47
B.	Supplementary tables	51

1. INTRODUCTION

A trading desk repricing a multi-asset book may solve the same PDE repeatedly, so the wall-clock cost of one solve affects the throughput available for valuation and risk analysis. Niesen and Wright’s option-pricing working paper reported an approximately 700s historical CPU result for one three-asset configuration.¹ The figure is useful historical context, not a contemporary performance baseline. In the implementation studied here, a Krylov exponential integrator spends over 70% of its time in sparse matrix–vector products (SpMV) that move far more bytes than they compute, and most of the remainder in an orthogonalization whose steps require reductions. Both are *communication* costs: data crossing the memory hierarchy, and workers waiting on collective operations [4, 19]. Communication-avoiding (CA) reformulations attack these costs—a matrix-powers kernel reuses an operator tile across several Krylov vectors [10, 22], and block orthogonalization reduces the number of synchronization phases [19, 7]—but pay for them with extra arithmetic and storage.

Whether that exchange pays is not a property of the algorithm alone: it depends on the operator, the grid, the implementation, and the machine. This thesis studies necessary opportunities for the exchange to pay and then tests them on the available hardware.

Three artifacts support that study. A **benchmark pricer** in C++23 implements five methods (Crank–Nicolson, two ADI variants, a one-shot matrix exponential, and a Krylov exponential integrator) on two three-asset European options and isolates temporal error for a fixed semi-discretization (Section 5). A **regime study** (Section 8) builds a dimensionless map of the CA design space. Its axes normalize demand by machine-dependent supply; their unit lines are opportunity indicators rather than promises of speedup or hardware portability. A **distributed CA exponential integrator** (Section 10) then evaluates those mechanisms on up to four GPUs across two nodes and asks whether their savings survive contact with a complete adaptive solver.

One design rule keeps the map non-circular: the coordinates are computed before a run from the problem size, sparsity pattern, spectrum, and independently measured machine constants; outcomes such as wall-clock, orthogonality loss, and achieved traffic do not enter them. The coordinates can therefore be tested as opportunity indicators, while the measured crossover remains an outcome rather than being fixed at one by definition.

The thesis proceeds as follows. Section 2 states the pricing problem and its discretization. Section 3 presents the Krylov exponential integrator, its cost structure, and the CA reformulations that target it. Section 4 describes the experimental platform and measurement methodology. Section 5 follows the historical Niesen–Wright option-pricing experiment as a temporal-integration control. Section 6 profiles the solver and places its kernels on a cache-aware roofline. Section 7 presents the OpenMP scaling study that motivates the two CA mechanisms. Section 8 develops the regime map and its numerical and measured results. Section 9 recomputes the coordinates for a GPU memory system and tests their transfer across two distinct GPU architectures. Section 10 builds the distributed integrator and measures topology-specific performance on a full solve. Section 11 names what the measurements leave open, and Section 12 concludes.

¹J. Niesen and W. M. Wright, *A Krylov Subspace Method for Option Pricing*, working paper, DOI: 10.2139/ssrn.1799124.

2. THE PRICING PROBLEM AND ITS DISCRETIZATION

The model problem is the 3-asset Black–Scholes PDE [6] in log-price coordinates, solved backward in pseudo-time τ from 0 (payoff) to T (price):

$$\frac{\partial u}{\partial \tau} = \frac{1}{2} \sum_{d,d'} \rho_{dd'} \sigma_d \sigma_{d'} \frac{\partial^2 u}{\partial x_d \partial x_{d'}} + \sum_d \left(r - \frac{1}{2} \sigma_d^2 \right) \frac{\partial u}{\partial x_d} - r u + b(\tau). \quad (2.0.1)$$

The three groups of terms recur throughout the thesis under their standard PDE names. The second-derivative terms are *diffusion* (volatility spreading value across the surface; the mixed $d \neq d'$ terms couple correlated assets). The first-derivative terms are *convection*, also called *advection* (the risk-neutral drift transporting value across the grid). The zeroth-order term $-ru$ is *reaction* (here, discounting at the risk-free rate).

Two payoffs are priced. The basket call,

$$u_0(x) = \max \left(\sum_d w_d e^{x_d} - K, 0 \right),$$

and the rainbow min-call,

$$u_0(x) = \max \left(\min_d e^{x_d} - K, 0 \right).$$

The log-price domain is unbounded, so truncating it to a finite computational box requires an artificial boundary closure. This closure is not arbitrary: it is chosen to match the known far-field financial behaviour of the payoff and thereby limit truncation artifacts. For the basket call, nodes beyond the upper boundary are deep in the money, meaning that the basket value is far above the strike and the terminal payoff is overwhelmingly likely to lie in the exercise region. There the option value is well approximated by the basket value minus the discounted strike. Expanding the discount factor $e^{-r\tau}$ to second order in τ and substituting this far-field form into the ghost-node stencil gives the polynomial forcing $b(\tau) = \mathbf{B}\mathbf{s}(\tau)$ with $\mathbf{s}(\tau) = [\tau^2/2, \tau, 1]^\top$. Its polynomial form both reflects the payoff asymptote and permits the exact autonomous $N+3$ augmentation used in Section 3. The rainbow case instead uses a zero-gamma closure—locally linear continuation of the option value—implemented through modified boundary stencils and hence requires no forcing augmentation.

Discretization is by second-order central differences on a uniform Cartesian grid with x_1 -fastest lexicographic ordering (grid points are numbered consecutively along x_1 first, then x_2 , then x_3), so every one-dimensional operator extends to the full grid as a three-fold Kronecker product with identity in the undifferentiated directions. The assembled $\mathbf{A}$ carries at most 19 nonzeros per row (a 7-point stencil plus 3×4 mixed-derivative diagonals). Its bandwidth is $O(N_1 N_2)$, set by the x_3 and mixed terms, which rules out banded direct solvers at scale. $\mathbf{A}$ splits as $\mathbf{A}_{\text{sym}} + \mathbf{A}_{\text{skew}}$ —diffusion, mixed, and reaction terms symmetric, convection skew-symmetric—so it is not symmetric in general. The measured eigenvalues have nonpositive real part, which is consistent with asymptotic decay, but the spectrum alone does not control transient amplification for a non-normal matrix; stability and approximation quality are therefore checked through the exponential action and residual diagnostics rather than inferred from the eigenvalues alone [20]. Table 1 lists the default model parameters used throughout.

Table 1.: Default model parameters.

Parameter	Value	Description
Spot prices S_0	(100, 100, 100)	Initial asset prices
Strike K	100	Option strike
Risk-free rate r	0.04	Continuously compounded
Maturity T	1.0 year	Fixed
Volatilities σ	(0.30, 0.35, 0.40)	Per-asset annual volatility
Correlations ρ	(0.50, 0.50, 0.50)	Off-diagonal pairs $(\rho_{01}, \rho_{02}, \rho_{12})$
Basket weights w	(1/3, 1/3, 1/3)	Equal-weight basket
Grid half-width α	2.85	Domain $\pm\alpha\sigma\sqrt{T}$ per dimension

Stiffness grows quadratically with refinement, and the discretization says why. The central second difference of the diffusion term on n interior points per axis has eigenvalues

$$\lambda_k = -\frac{2\sigma^2}{\Delta x^2} \left(1 - \cos \frac{k\pi}{n+1}\right), \quad k = 1, \dots, n,$$

so the extreme eigenvalue scales as $O(\Delta x^{-2}) = O(n^2)$ [20]. Measured on the assembled operator, the smallest eigenvalue moves from -24 at grid size $n = 11$ to -2924 at $n = 101$ (where $N = n^3$ is the total number of unknowns), consistent with that scaling. Over-solving in time on a coarse grid is therefore pointless. The approximately 700s result quoted in the introduction remains historical context from the Niesen–Wright option-pricing working paper, not the performance baseline for the present hardware.

3. THE KRYLOV EXPONENTIAL INTEGRATOR

For the basket payoff, spatial discretization gives the non-autonomous linear system

$$\frac{d\mathbf{u}}{d\tau} = \mathbf{A}\mathbf{u} + \mathbf{B}\mathbf{s}(\tau), \quad \mathbf{s}(\tau) = [\tau^2/2 \quad \tau \quad 1]^\top, \quad \mathbf{u}(\tau_0) = \mathbf{u}_0. \quad (3.0.1)$$

The three-column matrix $\mathbf{B}$ contains the coefficients of the polynomial boundary forcing described in Section 2. Since $\mathbf{s}'(\tau) = \mathbf{K}\mathbf{s}(\tau)$ for the nilpotent shift

$$\mathbf{K} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix},$$

the forcing can be absorbed exactly into an autonomous augmented system. Define

$$\tilde{\mathbf{A}} = \begin{bmatrix} \mathbf{A} & \mathbf{B} \\ \mathbf{0} & \mathbf{K} \end{bmatrix} \in \mathbb{R}^{M \times M}, \quad M = N + 3, \quad \tilde{\mathbf{v}} = \begin{bmatrix} \mathbf{u}_k \\ \mathbf{s}(\tau_k) \end{bmatrix} \in \mathbb{R}^M. \quad (3.0.2)$$

Then the first N components of $e^{h\tilde{\mathbf{A}}}\tilde{\mathbf{v}}$ equal $\mathbf{u}(\tau_k + h)$, while the last three components equal $\mathbf{s}(\tau_k + h)$. The rainbow payoff has no forcing matrix in the implementation: for that case $M = N$, $\tilde{\mathbf{A}} = \mathbf{A}$, and $\tilde{\mathbf{v}} = \mathbf{u}_k$.

The Krylov approximation proceeds via the Arnoldi iteration on $\tilde{\mathbf{A}}$ with starting vector $\tilde{\mathbf{v}}$, producing an orthonormal basis $\mathbf{V}_m$ and an upper Hessenberg matrix $\bar{\mathbf{H}}_m \in \mathbb{R}^{(m+1) \times m}$ [24, 18]. Dropping the last row gives $\mathbf{H}_m \in \mathbb{R}^{m \times m}$, and the approximation reads

$$e^{h\tilde{\mathbf{A}}}\tilde{\mathbf{v}} \approx \beta \mathbf{V}_m e^{h\mathbf{H}_m} \mathbf{e}_1, \quad \beta = \|\tilde{\mathbf{v}}\|. \quad (3.0.3)$$

Algorithm 1: Adaptive Krylov Exponential Integrator for the Basket Polynomial Forcing

Input: Matrix $\mathbf{A} \in \mathbb{R}^{N \times N}$, forcing coefficients $\mathbf{B} \in \mathbb{R}^{N \times 3}$, initial condition $\mathbf{u}_0$, final time T , time-step $\Delta\tau$, absolute defect tolerance tol , maximum dimension $m_{\max}$

Output: Solution snapshots $\{\mathbf{u}_k\}_{k=0}^K$ at $\tau_k = k \Delta\tau$

```

1   $\mathbf{K} \leftarrow \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}; \quad \tilde{\mathbf{A}} \leftarrow \begin{bmatrix} \mathbf{A} & \mathbf{B} \\ \mathbf{0} & \mathbf{K} \end{bmatrix}; \quad M \leftarrow N + 3;$ 
2   $\tau \leftarrow 0;$ 
3  for  $k = 0, 1, \dots, K - 1$  do
4       $h \leftarrow \min(\Delta\tau, T - \tau);$ 
5       $\tilde{\mathbf{v}} \leftarrow [\mathbf{u}_k^\top, \mathbf{s}(\tau)^\top]^\top$  // Eq. (3.0.2)
6       $m \leftarrow 1; \quad \eta_m \leftarrow \infty;$ 
7      /* Adaptive Krylov loop */
8      while  $m \leq \min(M, m_{\max})$  do
9          Run  $m$  steps of Arnoldi on  $(\tilde{\mathbf{A}}, \tilde{\mathbf{v}}) \Rightarrow \mathbf{V}_{m+1}, \bar{\mathbf{H}}_m$  // Algorithm 2
10         Store  $\bar{h}_{m+1,m} \leftarrow [\bar{\mathbf{H}}_m]_{m+1,m}$  // sub-diagonal entry
11          $\mathbf{H}_m \leftarrow \bar{\mathbf{H}}_m(1:m, 1:m)$  // drop last row
12          $\beta \leftarrow \|\tilde{\mathbf{v}}\|;$ 
13          $\mathbf{f} \leftarrow e^{h\mathbf{H}_m} \mathbf{e}_1$  // small  $m \times m$  matrix exponential
14          $\mathbf{w} \leftarrow \beta \mathbf{V}_m \mathbf{f}$  // Krylov approximation in full space
15         /* Endpoint residual/defect indicator */
16          $\eta_m \leftarrow \beta |\bar{h}_{m+1,m}| \|[\mathbf{f}]_m\|;$ 
17         if  $\eta_m < \text{tol}$  then
18             break;
19         end
20          $m \leftarrow m + 1;$ 
21     end
22      $\mathbf{u}_{k+1} \leftarrow \mathbf{w}(1:N)$  // extract first  $N$  components
23      $\tau \leftarrow \tau + h;$ 
24 end
    
```

Algorithm 2: Arnoldi Iteration

Input: Matrix $\tilde{\mathbf{A}} \in \mathbb{R}^{M \times M}$, vector $\tilde{\mathbf{v}} \in \mathbb{R}^M$, subspace dimension m , breakdown tolerance $\text{tol}_{\text{break}}$

Output: Orthonormal basis $\mathbf{V}_{m+1} \in \mathbb{R}^{M \times (m+1)}$, upper Hessenberg $\bar{\mathbf{H}}_m \in \mathbb{R}^{(m+1) \times m}$

```

1   $\beta \leftarrow \|\tilde{\mathbf{v}}\|; \quad \mathbf{v}_1 \leftarrow \tilde{\mathbf{v}}/\beta;$ 
2  for  $j = 1, \dots, m$  do
3       $\mathbf{w} \leftarrow \tilde{\mathbf{A}} \mathbf{v}_j;$ 
4      for  $i = 1, \dots, j$  do
5          // Modified Gram-Schmidt
6           $h_{i,j} \leftarrow \mathbf{v}_i^\top \mathbf{w};$ 
7           $\mathbf{w} \leftarrow \mathbf{w} - h_{i,j} \mathbf{v}_i;$ 
8      end
9       $h_{j+1,j} \leftarrow \|\mathbf{w}\|;$ 
10     if  $h_{j+1,j} < \text{tol}_{\text{break}}$  then
11         break // Invariant subspace found (happy breakdown)
12     end
13      $\mathbf{v}_{j+1} \leftarrow \mathbf{w}/h_{j+1,j};$ 
14 end
    
```

Algorithm 1 gives the full adaptive procedure, and Algorithm 2 the Arnoldi iteration it calls.

The scalar η_m in Algorithm 1 has a precise but limited meaning. For the projected trajectory $\tilde{\mathbf{y}}_m(t) = \beta \mathbf{V}_m e^{t\mathbf{H}_m} \mathbf{e}_1$, the endpoint defect is parallel to $\mathbf{v}_{m+1}$ and has norm

$$\eta_m(h) = \beta |\bar{h}_{m+1,m}| |\mathbf{e}_m^\top e^{h\mathbf{H}_m} \mathbf{e}_1|.$$

The implementation compares this absolute endpoint indicator with 10^{-8} for each Krylov action. It is not, without an additional stability estimate or integration of the defect, an upper bound on the state error, a global time-integration error, or an option-price error. Comparisons with an independent exponential-action referee provide the separate solution-accuracy check used later in the thesis.

3.1. COST STRUCTURE AND THE SEQUENTIAL DEPENDENCY

The dominant cost of the integrator is the Arnoldi iteration. Each step j requires one sparse matrix–vector product $\mathbf{w} = \tilde{\mathbf{A}}\mathbf{v}_j$, costing $O(N)$, and orthogonalization against all previous vectors, costing $O(jN)$. Crucially, step j cannot begin until $\mathbf{v}_j$ is available, which requires step $j-1$ to have completed. This creates a strictly sequential data dependency across the Krylov dimension m . The individual SpMV and inner products within each step can be parallelized, but this inner parallelism scales poorly: the SpMV is bandwidth-bound, and every inner product in the modified Gram–Schmidt (MGS) loop is a global reduction. These are the two *communication* walls (Table 2), and they are different in kind.

Table 2.: The two kernels, their bounds, and the CA mechanism each motivates.

Kernel	Bound	Communication	CA mechanism
SpMV	bandwidth (memory hierarchy)	vertical	matrix powers / cache blocking
Gram–Schmidt	synchronization (reductions)	horizontal	s -step / block orthogonalization

3.2. COMMUNICATION-AVOIDING REFORMULATIONS

To remove an off-by-one ambiguity, q denotes the number of columns in a block throughout the mathematical discussion. A q -column monomial block is $\mathbf{Z}_q = [\mathbf{v}, \mathbf{A}\mathbf{v}, \dots, \mathbf{A}^{q-1}\mathbf{v}]$ and requires $q-1$ operator applications. The archived CPU instrument uses the legacy field `s_max` for the largest polynomial degree, so its corresponding block has $q_{\max} = \text{s_max} + 1$ columns. The distributed executable instead labels its block-column input `s`; that name is retained in tables that reproduce command-line output, where `s` = q .

The CA-Arnoldi reformulation [19, 10, 7] constructs one such block per communication phase. The *matrix-powers kernel* [22] reuses an operator tile across the $q-1$ applications, attacking the vertical wall. The block is then orthogonalized at once—here by CholeskyQR2 [27]—which reduces the number of synchronization phases relative to MGS.

The condition-number test used by the implementation requires a qualification that is important for every result below. For a tall matrix $\mathbf{Z} \in \mathbb{R}^{M \times q}$, the cited CholeskyQR2 analysis includes the dimension-dependent sufficient condition

$$\delta = 8 \kappa_2(\mathbf{Z}) \sqrt{Mqu + q(q+1)u} \leq 1, \quad (3.2.1)$$

in addition to its other hypotheses. The simpler line $\kappa_2(\mathbf{Z}) \leq u^{-1/2}$ describes a practical breakdown scale for the normal equations; it is not a dimension-free sufficient theorem. For the production basket grid, $M = 61^3 + 3 = 226,984$ and $q = 4$, so (3.2.1) limits $\kappa_2(\mathbf{Z})$ to approximately 1.25×10^4 . Substitution of the measured 6.0×10^6 gives $\delta \approx 482$. Because (3.2.1) is sufficient rather than necessary, this value makes the theorem inconclusive for the production block; it does not diagnose instability. The distributed Gram matrix also introduces all-reduce rounding beyond the serial formulation.

Throughout the thesis, $u^{-1/2}$ is therefore called the *implemented conditioning guard* or *heuristic line*. Its operational role is to detect rapidly growing block conditioning and trigger a narrower block before normal-equations breakdown. Numerical acceptance is based on the guard together with Cholesky success, measured orthogonality, and final-state agreement with an independent referee. This separates the theorem’s sufficient guarantee from the direct evidence available for the tested implementation. The regime study then measures how the monomial block conditioning grows with q and how that growth affects the width selected by the guard.

An alternative not pursued here is the rational Krylov subspace [5], which replaces the sequential polynomial recurrence with m independent shifted solves. Its parallelism is attractive, but each solve is itself a sparse linear system whose iterative solution re-imports both communication walls, and the near-optimal pole selection adds problem-specific tuning. The s -step route was chosen because its costs are analyzable in machine terms—which is precisely what the regime map requires.

4. EXPERIMENTAL PLATFORM AND METHODOLOGY

Every CPU result in this thesis was gathered on a single bare-metal workstation (“puffin”): an AMD Ryzen Threadripper 3960X, 24 cores in a single socket and a single NUMA node, organized as 8 *core complexes* (CCX)—clusters of 3 cores that share one 16 MiB slice of L3 cache (128 MiB total). Per-core caches are 32 KiB L1d and 512 KiB L2, nominal maximum clock is 3.8 GHz, and main memory is 4-channel DDR4-3200 with a theoretical 102.4 GB/s roof. Each core also exposes a second hardware thread via simultaneous multithreading (SMT). The machine also hosts an NVIDIA RTX 3090 (24 GB), which is one half of the GPU pair below. The machine’s derived constants live in a single header (`include/machine.hpp`) under the key `amd-3960x`, and every predicted coordinate in the regime study is computed from them.

THE GPU PLATFORM. The GPU study of Section 9 adds a second machine, “syngé”: a two-node cluster carrying a pair of NVIDIA Tesla V100-PCIE-16GB cards per node (four in total), the two cards within a node joined by PCIe across a cross-socket hop with no NVLink, and the two nodes by a 100 Gb/s Omni-Path fabric (over which the NCCL collective library selects its InfiniBand-verbs transport). Syngé’s V100 and puffin’s RTX 3090 form a useful but not controlled architectural contrast. Their measured bandwidths and nominal L2 capacities are similar, while their FP64 peaks differ by about 11×; they also differ in SM organization, cache hierarchy, clocks, memory technology, ECC behavior, schedulers, and launch behavior. Conclusions from the pair are therefore kernel- and architecture-specific rather than causal estimates of FP64 throughput alone. Their constants live in a separate header (`include/gpu_machine.hpp`), typed distinctly from the CPU constants so that a CPU value reaching a GPU coordinate is a compile error rather than a plausible wrong number, and are calibrated per card on an idle device rather than read from datasheets.

WORKSTATION SELECTION. A cloud arm was seriously considered, and its attractions are real: cloud providers offer server-grade hardware at reasonable hourly cost where puffin is commodity desktop silicon, better hardware would open larger problem configurations, and in a commercial setting these

services are exactly where a desk would run its own research. It was dropped anyway, on two grounds. Contention: timing variance that looks internally consistent is simultaneously evidence of memory-bound behavior and of noise from co-tenants, and from inside the guest the two are not separable—so economic feasibility comes bundled with uninterpretable measurements. Topology, the sharper objection: the hypervisor reports a socket and a private last-level cache slice *per vCPU*, a fiction of the virtualization rather than the silicon. Both coordinates of the regime map are functions of real cache geometry (one is a cache-capacity ratio, the other is set by how many cache-domain boundaries a reduction tree crosses), so neither is knowable on an invented topology, and the map cannot be drawn there at all. The study therefore requires an observable physical topology and explicit contention controls; on the shared cluster, idle-device gating supplies the corresponding safeguard. Bare metal removes the hypervisor but not OS scheduling noise, so timed quantities are reported as medians with the stated spread statistic over repeated trials. On a boost-enabled processor, the median represents a typical repeat, whereas the minimum can reflect a transient boost state.

REPRODUCIBILITY. The computational workflow is organized so that each reported result can be traced from implementation to figure. The repository provides the CMake build, C++23 experiment programs, centralized machine descriptions, run scripts, and a named plotting script for each figure with inline dependency metadata. Experimental parameters and timing protocols are recorded alongside the corresponding tables and captions, while the executables use the same audited machine constants that define the regime coordinates. Additional run metadata, raw measurements, and profiler outputs for a specific experiment are available from the author on request.

5. HISTORICAL NIESEN–WRIGHT TIME-INTEGRATOR COMPARISON

Five methods are compared. Three are time-steppers over a fixed number of substeps: Crank–Nicolson ($\theta = 0.5$, fully implicit, second order, one sparse LU factorization per run), and two alternating-direction-implicit splittings—Douglas–Rachford [11], with three direction-split solves per step, and Hundsdorfer–Verwer [20, 21], a Douglas–Rachford predictor with a corrector. The remaining two exponentiate the operator. The matrix exponential (ME) applies the exponential action by scaling and a truncated Taylor recurrence [16, 2] with an ∞ -norm early exit, choosing its own substep count. The Krylov subspace exponential integrator (KSM-EI) is the method of Section 3, growing the basis until its endpoint defect indicator meets the tolerance.

The benchmark follows the experimental comparison in the Niesen–Wright option-pricing working paper named in the introduction: log–log plots of semi-discrete ODE error against CPU time for all five methods, across both option types and both grid sizes (Figure 1).

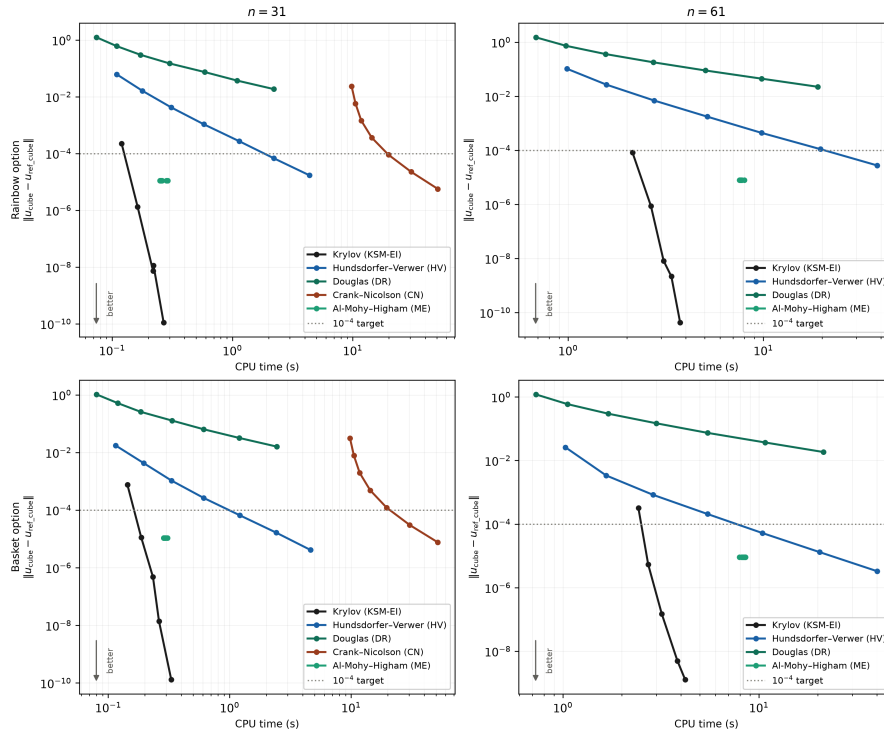


Figure 1.: Semi-discrete ODE error vs. CPU time for the five methods, across both payoffs and grid sizes, following the historical Niesen–Wright option-pricing comparison. The Krylov integrator is fastest in the $\sim 10^{-4}$ temporal-accuracy band for these runs; Crank–Nicolson’s non-monotone stiff-mode ringing ($R(-\infty) = -1$) is visible.

THE REFEREE. The measurement is against a high-accuracy solution of the same semi-discrete ODE, not an analytic or independent financial price. Before each option type is benchmarked, a matrix-exponential referee is computed at maximum Taylor degree $m = 55$ and $\theta = 9.9$, giving $s = \lceil T \|\tilde{\mathbf{A}}\|_1 / \theta \rceil$ internal substeps at a tolerance of $2^{-53} \approx 1.1 \times 10^{-16}$. This isolates temporal-integration error, which is the purpose of the experiment. It does not include spatial discretization, finite-domain truncation, boundary-condition, or interpolation error and therefore does not establish total price or Greek accuracy. The reported ODE error is the Euclidean norm over the $9 \times 9 \times 9$ grid neighborhood centered on the spot rather than the value at a single node.

ROLE OF THE BENCHMARK. This section serves as a control on the temporal integrators and their implementation. At $n = 15$ every method reaches the same semi-discrete floor, so further temporal work buys nothing on that grid. The comparison motivates studying the Krylov implementation at larger states; establishing contemporary pricing competitiveness would additionally require equal-total-accuracy comparisons with optimized GPU ADI and dimension-reducing methods.

6. PROFILE AND ROOFLINE

At $n = 61$ (226,981 unknowns, 4.23M nonzeros, ≈ 49 MB in compressed sparse column form) over 100 steps, the KSM-EI solve is overwhelmingly SpMV and Gram–Schmidt; the dense $\exp(\mathbf{H}_m)$ is marginal (Table 3). Gram–Schmidt’s share grows with the grid because the Krylov dimension m grows and MGS is quadratic in m . The *other* column collects everything outside the three instrumented kernels: vector updates, norm computations for the endpoint defect indicator, and per-step bookkeeping.

Table 3.: Kernel shares of KSM-EI solve time (basket, tolerance 10^{-8}). $\bar{m}$ is the mean Krylov dimension; *other* is uninstrumented work (vector updates, error estimation, bookkeeping).

Grid	$\bar{m}$	SpMV	Gram–Schmidt	dense expm	other
$n = 31$	7.60	82.9%	11.7%	0.40%	4.9%
$n = 53$	8.42	77.7%	16.8%	0.34%	5.1%
$n = 65$	9.24	76.0%	18.0%	0.21%	5.9%
$n = 73$	9.89	75.2%	18.8%	0.15%	5.9%
$n = 89$	11.34	70.3%	23.8%	0.08%	5.8%

Both kernels sit far below the compute ceiling. The cache-aware roofline [26] shows why, and where the CA opportunity is (Figure 2, Table 4). Single-core roofs on puffin, all measured: DRAM 20.9 GB/s, L3 58.2 GB/s, L2 88.6 GB/s, compute peak 68.1 GFLOP/s (an FMA loop).

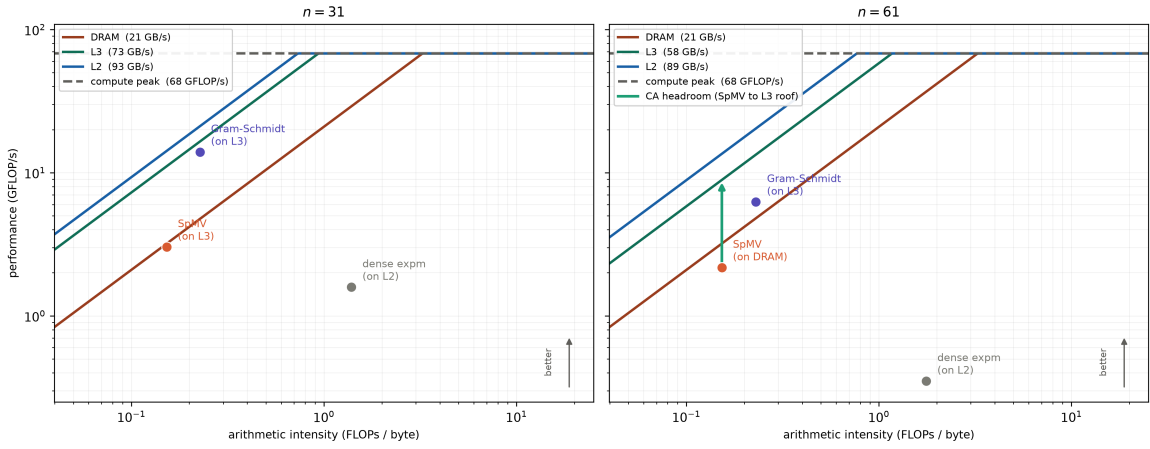


Figure 2.: Cache-aware roofline for the two hot kernels at $n = 31$ and $n = 61$. At $n = 61$ the SpMV has spilled to DRAM, the vertical distance from the measured point to the L3 roof at the same arithmetic intensity is exactly the traffic a matrix-powers reformulation would recover.

Table 4.: Measured kernel positions on the roofline. AI is arithmetic intensity.

Grid	Kernel	AI (FLOP/B)	Achieved (GFLOP/s)	Resident tier
$n = 31$	SpMV	0.153	3.02	L3
$n = 31$	Gram–Schmidt	0.227	13.91	L3
$n = 61$	SpMV	0.153	2.17	DRAM
$n = 61$	Gram–Schmidt	0.230	6.24	L3

The attainable gain from moving a kernel one tier up the hierarchy is

$$\text{gain} = \frac{\min(BW_{\text{fast}} \cdot AI, \text{peak})}{\min(BW_{\text{slow}} \cdot AI, \text{peak})}, \quad (6.0.1)$$

which collapses to the bandwidth ratio only while *both* roofs are still sloped. It must bend down once the fast roof reaches the compute ceiling—which the $s \times$ -higher arithmetic intensity of a matrix-powers kernel can actually reach. Quoting a flat bandwidth ratio as “the CA speedup” overstates the ceiling.

7. THE OPENMP SCALING STUDY

The scaling study measures how the two hot kernels of one Arnoldi cycle respond to added cores—it does not measure solution accuracy. The thesis of the experiment is that the two kernels fail to scale for two *different* reasons, each motivating a distinct CA mechanism (Table 2), this split is what the regime analysis of Section 8 later turns into two independent dimensionless axes.

7.1. INSTRUMENT DESIGN

To make both ceilings visible and attributable, the harness freezes the Krylov dimension at $m = 8$ with the convergence check disabled, so the Gram–Schmidt/SpMV work ratio does not drift and any runtime change across the core count P is attributable to hardware rather than a changing algorithm (the genuine $m(n)$ growth is measured separately). The SpMV stores the matrix in compressed sparse row (CSR) format—each row’s nonzeros held contiguously—and is hand-rolled, i.e. written for this study rather than taken from a library, so that the row partition and the reduction strategy are fully under the experiment’s control: each thread is the sole writer of a contiguous, disjoint slice of the output vector, with partition seams aligned to the cache line. The harness runs MGS with parallel global reductions, exposing the synchronization cost directly, and reports per-kernel timings so a single sweep yields SpMV’s plateau and Gram–Schmidt’s roll-off on one axis. Runs pin threads with `OMP_PLACES=cores` and `OMP_PROC_BIND=close`, so each CCX (Section 4) fills before the next engages and per-CCX L3 bandwidth steps appear as discrete risers. The swept core counts are CCX-boundary aligned, and SMT lanes are excluded so no core is timed while sharing its pipelines with a second thread. Each point is the median of five to seven timed repeats after a warm-up, with the inter-quartile range as the error bar.

Two initialization arms form a controlled A/B experiment on memory placement: arm A fills the matrix from thread 0 (first-touch parks it on one memory placement), arm B has each thread first-touch exactly the row partition it will later compute on. The two arms turn out to be indistinguishable—for a reason that is itself the finding, taken up in Section 7.3. Figures 3 and 4 show the strong- and weak-scaling deliverables.

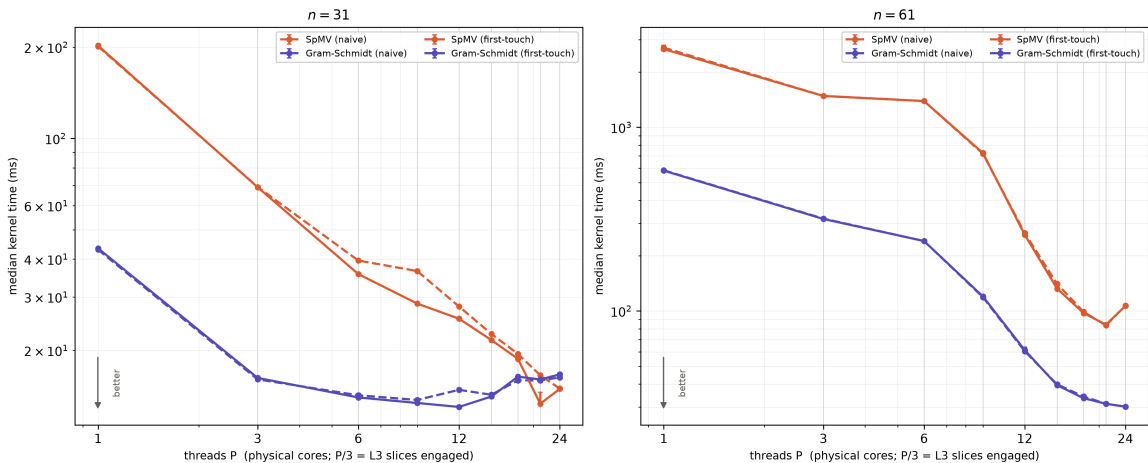


Figure 3.: Per-kernel strong scaling at $n = 31$ and $n = 61$, both initialization arms. SpMV shows a plateau at $n = 31$ and a superlinear transition at $n = 61$; Gram–Schmidt shows the synchronization staircase.

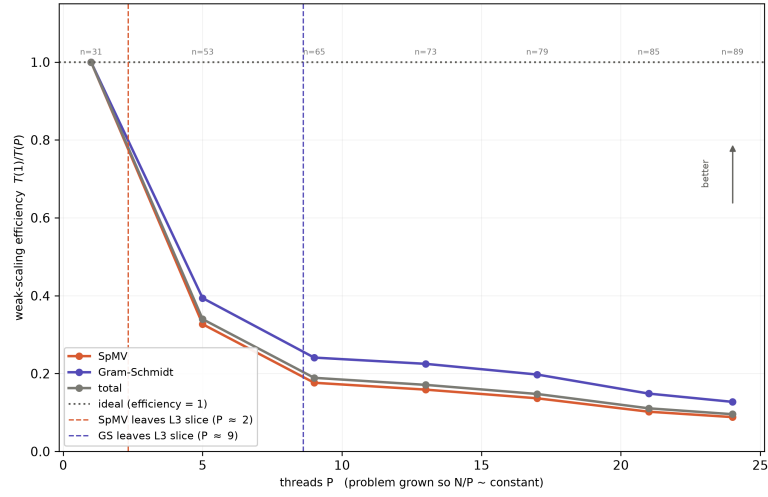


Figure 4.: Per-kernel weak-scaling efficiency $T(1)/T(P)$ with $n(P) = n_1 P^{1/3}$ rounded odd, cache-tier crossings annotated.

7.2. THE QUARANTINED $m(n)$ GROWTH

The harness freezes m so that the scaling curves measure hardware; the genuine growth of the Krylov dimension with the grid is measured separately and reported as an independent numerical-analysis finding (Figure 5). The mean $\bar{m}$ per step grows from 7.6 at $n = 31$ to 11.3 at $n = 89$ —slow growth, but enough to drive Gram–Schmidt’s quadratic share upward (Table 3), and the input to the small- m safety argument of Section 8.4.

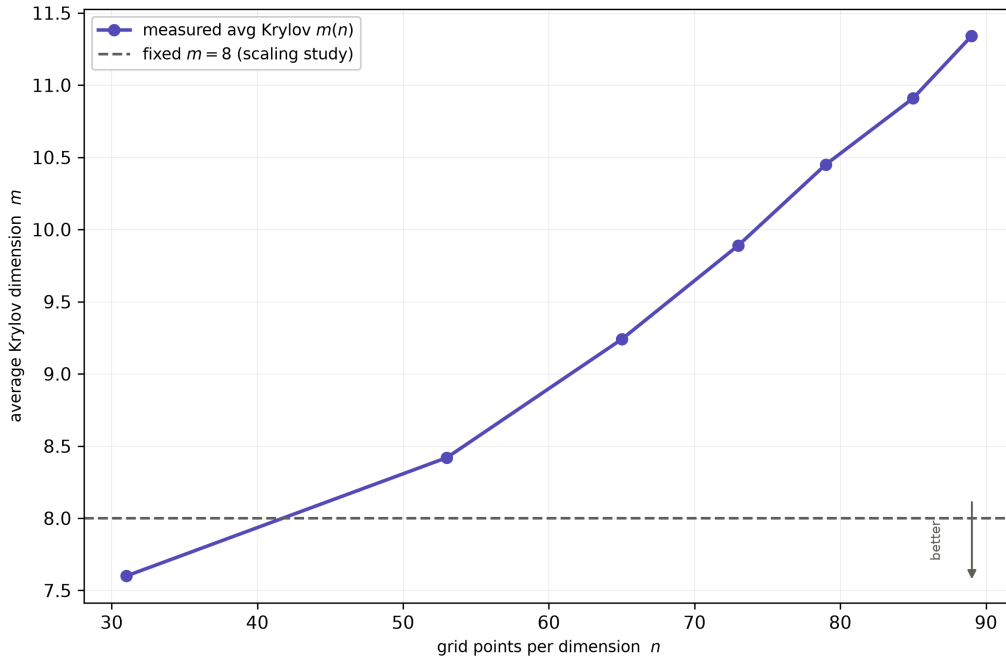


Figure 5.: The $m(n)$ Krylov-growth curve, measured with the convergence check enabled and quarantined from the scaling instrument, which pins $m = 8$.

7.3. THE LOCALITY CONTROL

At $n = 61$ the SpMV runtime drops *superlinearly* ($36.7\times$ on 24 cores) with a knee at $P = 12$ (Figure 3). The candidate explanation is cache residency: once each CCX’s share of the matrix ($3 \times 52.8/P$ MiB) fits in its own 16 MiB L3 slice and stays resident across the repeated SpMV, reads come from L3 instead of DRAM. Quantitatively this checks out—the implied aggregate bandwidth crosses the independently measured socket DRAM ceiling (~ 95 GB/s) at $P = 12$, ruling out DRAM alone and supporting service from the cache hierarchy.

The initialization arms *cannot* test this: the 3960X is single-NUMA, and Zen 2 L3 is a per-CCX victim cache, so L3 residency is set by which core *accesses* a row, not by which thread first-touched the page. Both arms use the same row partition and produce identical curves. The real controlled experiment is over the *compute schedule*, which changes only which rows each core reads (Table 5, Figure 6).

Table 5.: The locality control at $n = 61$: observed SpMV behavior per compute schedule (speedup at $P = 24$; implied aggregate bandwidth at $P = 12$).

Schedule	What it does	Observed behavior
block	contiguous band per core, fixed across SpMVs	$36.7\times$, knee at $P = 12$, 176 GB/s
cyclic	cache-line blocks dealt round-robin, fixed	$7.0\times$, no knee, 36 GB/s
rotate	contiguous bands, rotating by one each SpMV	$6.6\times$, no knee, 66 GB/s

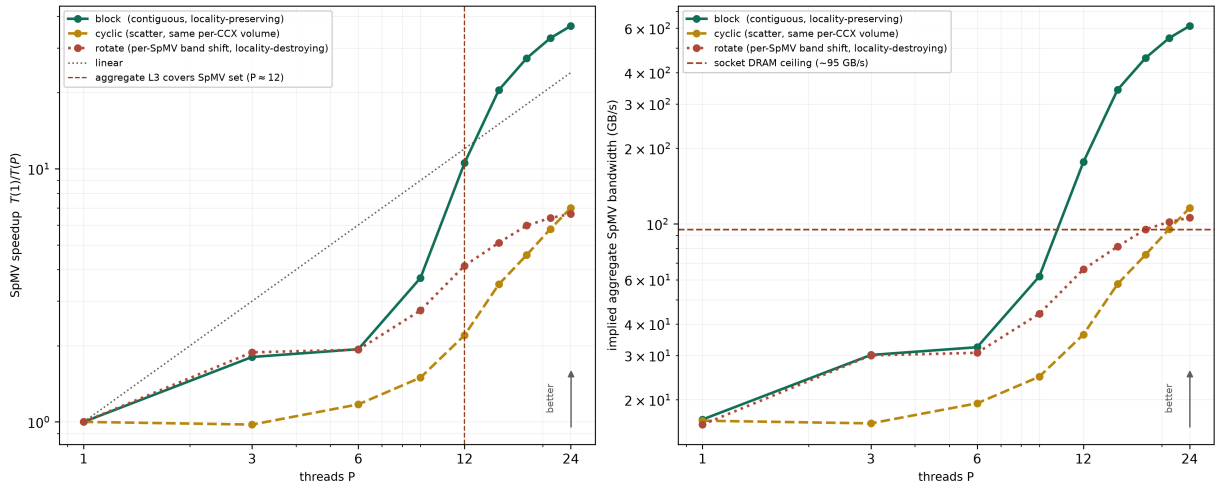


Figure 6.: The locality control. Only **block** carries an aggregate bandwidth DRAM cannot supply, so the transition is genuinely L3 residency, not a scheduling coincidence.

The **cyclic** schedule was predicted to keep the transition, since it leaves per-CCX byte *volume* unchanged; it lost the transition instead, collapsing to the **rotate** curve ($7.0\times$ vs. $6.6\times$, against **block**’s $36.7\times$). **Contiguity, not volume, is what keeps a slice resident**—exactly the property a matrix-powers tiling exploits, and exactly the property the scattered arm of the vertical sweep (Section 8.9) destroys on purpose. The identical arm-A/arm-B result is thereby the control that establishes the transition as a locality effect—the “before” that a matrix-powers reformulation would improve on.

8. REGIME ANALYSIS

The scaling study says the baseline solver is memory-bound and synchronization-bound. It does not say whether a CA reformulation would *help*, or where. The regime analysis organizes that question with a dimensionless map whose axes represent the two communication mechanisms. The unit line on each axis is a nominal opportunity boundary: it marks demand comparable with supply, not a universal performance crossover. A synthetic instrument traverses the map, and the real Black–Scholes operator is placed on it for comparison with measured outcomes.

The synthetic instrument generalizes the discretized operator of Section 2 into a family with independently controllable knobs. Its backbone is a *Kronecker sum*: one tridiagonal advection–diffusion operator $\mathbf{T} \in \mathbb{R}^{n_1 \times n_1}$ (the same three-point stencil as Section 2, with diffusion scale σ and advection strength γ) is extended to the d -dimensional grid by

$$\mathbf{A}_d = \mathbf{T} \oplus \cdots \oplus \mathbf{T} = \sum_{i=1}^d \mathbf{I} \otimes \cdots \otimes \underbrace{\mathbf{T}}_{i\text{-th factor}} \otimes \cdots \otimes \mathbf{I} \in \mathbb{R}^{n_1^d \times n_1^d},$$

which is exactly how a separable PDE discretization acts: the same one-dimensional stencil applied along each axis in turn [15]. Two optional terms then break that clean structure on demand, mirroring the two ways the real operator of (2.0.1) departs from a textbook Laplacian. The *correlation cross term* adds $c(\mathbf{D} \otimes \mathbf{D})$, the discrete mixed second derivative coupling one axis pair—the role the ρ_{dd} terms play in the PDE. It is optional because separability is itself a variable under study: with the term off, the operator factors axis by axis and its spectrum is known in closed form; with it on, that factorization is broken in a controlled way. The *symmetric scatter permutation* relabels the grid unknowns, replacing $\mathbf{A}$ by $\mathbf{PAP}^\top$ for a permutation matrix $\mathbf{P}$: the same operator stored in a different memory layout, which destroys the banded structure a cache-blocked kernel relies on while provably changing no numerical property (gate C2 below).

The knobs are chosen so that each axis of the map moves *independently*: the grid size $N = n_1^d$ sets the Arnoldi working set and therefore R_v ; the spectrum controls $(\sigma, \mu, \gamma, \rho)$ set the basis conditioning and therefore the width admitted by the implemented guard, without touching the memory footprint; and the scatter knob changes the memory-access pattern at fixed spectrum. One operator family can therefore move one modeled coordinate while holding the other fixed, which makes it an instrument rather than just a test matrix.

8.1. THE COORDINATES

Table 6.: The two dimensionless coordinates.

Axis	Definition	Mechanism it governs	Scales like
R_v	Arnoldi working set / aggregate last-level cache	vertical: matrix-powers tiling, avoiding DRAM traffic	$\sim 1/P$
R_h	reduction cost / compute per core between reductions	horizontal: s -step, cutting the reduction count	$\sim P \log P$

Each coordinate is built as demand over supply. R_v divides the bytes occupied by the Arnoldi working set by the last-level-cache capacity available to hold it; R_h divides the latency of a global reduction by the useful computation between reductions. The unit lines $\theta_v = \theta_h = 1$ mark nominal equality of those modeled quantities. Below a line, the corresponding resource is not an evident first-order opportunity;

above it, there is demand that a CA mechanism might reduce. Neither statement establishes where wall-clock crosses over, because associativity, reuse distance, halo work, launch costs, overlap, and topology are absent from the two ratios. The four map regions are therefore labelled by *opportunity*—neither, horizontal, vertical, or both—rather than by a prediction that a mechanism pays. The Krylov dimension m is a modulator: it changes the working set and therefore R_v , but two machines with the same m can sit on opposite sides of the nominal line.

R_h ’s numerator needs a magnitude, not just a shape in P , or the nominal line cannot be evaluated. A calibration instrument measures it as an isolated scalar all-reduce with no application work; the scalar is the payload. The model fitted to it is

$$t_{\text{reduce}}(P) = t_{\text{intra}} \cdot \text{intra levels}(P) + t_{\text{cross}} \cdot \min(\text{cross levels}(P), L^*). \quad (8.1.1)$$

On puffin: $t_{\text{intra}} = 120.8 \text{ ns}$, $t_{\text{cross}} = 652.8 \text{ ns}$ (a cross-CCX tree level costs $5.4\times$ an intra-CCX one), and $L^* = 2.20$ —past roughly two cache-domain crossings, subtree concurrency hides the rest. Measured tree all-reduce latency runs from 9.6 ns at $P = 1$ to $1.70 \mu\text{s}$ at $P = 24$ ($R^2 = 0.98$ against the model). The cost is set by how many cache-domain boundaries the tree crosses, not by core count—which is precisely why this coordinate cannot be measured on a virtualized topology (Section 4).

8.2. THE INSTRUMENT AND ITS GATES

Before any timed experiment, a single-threaded control program verifies the instrument’s arithmetic claims (Table 7). Nothing in it is timed and no result depends on the host, so its verdicts are properties of the operator, not the machine. Each control is either a *gate*—a claim the rest of the study assumes, whose failure invalidates everything downstream—or a *finding*, a measured result reported in its own right.

Table 7.: The controls. A gate failure is fatal; a finding is a result.

Control	Claim under test	Status
C1	the Kronecker-sum spectrum is analytic and correct	gate
C2	the scatter knob is an exact similarity transform (so a pure- R_v dial exists)	gate
C3	measured m obeys the Hochbruck–Lubich spectral bound [18]	gate (Hermitian PSD)
C3b	a spectral shift leaves m fixed	finding
C4	κ of the matrix-powers basis equals the row-scaled Vandermonde from the spectrum	gate when normal
C5	relation of the implemented $u^{-1/2}$ guard to observed factorization behaviour	empirical finding
C5b	worst-case block count over the physical ensemble (“small- m safety”)	finding
C6	where the operator is non-normal, does the spectral prediction survive?	finding

C2 is load-bearing. The scatter knob applies a symmetric permutation $\mathbf{PAP}^\top$ with $\mathbf{P}$ a permutation matrix; since $\mathbf{P}$ is orthogonal, $\mathbf{PAP}^\top = \mathbf{PAP}^{-1}$ is a similarity transform, and similar matrices share their spectrum exactly [15, 25]. The permutation therefore moves bytes—which rows sit near which—and provably nothing else. If it moved the spectrum, the “pure R_v line” would be an illusion and every vertical result would be confounded. The sweep runs in five phases—gate, confound, m -range, non-normal, and finally the real discretized Black–Scholes operator with its actual discounted payoff,

swept across grids. That last phase is *transfer by measurement*: it lands the real operator on the same (R_v, R_h) plane as the synthetic instrument.

8.3. BASIS CONDITIONING AND THE CONFOUND CONTROL

The regime instrument records $s_{\max}$, the largest polynomial degree admitted by its implemented $u^{-1/2}$ conditioning guard; the corresponding block has $q_{\max} = s_{\max} + 1$ columns. A row-scaled Vandermonde construction [25] predicts this *heuristic* degree from the spectrum and the experiment compares that prediction with the guard. Where a closed-form spectrum exists, the two integers agree on 20 of 21 sampled runs; the exception is the most non-normal point in that sweep ($\gamma = 0.4$, $\kappa(X) = 7.7 \times 10^8$) and differs by one degree. This descriptive agreement rate characterizes the sampled predictor–guard relationship; theorem-level stability and population-level accuracy are separate questions. The real Black–Scholes points differ by at most one degree over the measured grids.

The obvious confound is that the guard-admitted $s_{\max}$ might just be tracking the Krylov dimension m . The algebra says the block conditioning should not: for a normal operator $\mathbf{A} = \mathbf{X}\mathbf{\Lambda}\mathbf{X}^*$, the matrix-powers block factors as

$$[\mathbf{v}, \mathbf{A}\mathbf{v}, \dots, \mathbf{A}^s\mathbf{v}] = \mathbf{X} \operatorname{diag}(\mathbf{X}^*\mathbf{v}) \mathbf{V}_s(\lambda_1, \dots, \lambda_N),$$

where $\mathbf{V}_s$ is the Vandermonde matrix in the eigenvalues—so the block’s conditioning is governed by the eigenvalues and the starting vector’s eigen-coefficients, and m appears nowhere [25, 7]. The experiment is consistent with that separation: with the spectrum pinned and m moved through 5–12 (via the step size and tolerance), $s_{\max}$ stays flat at 7; with m pinned at 5 and the spectrum scaled and shifted, $s_{\max}$ ranges from 4 to 14. This controls the conditioning confound; downstream numerical behaviour is assessed by the separate orthogonality and solution diagnostics.

8.4. THE REDUCTION LEDGER AND SMALL- m SAFETY

The comparison is MGS against CholeskyQR2 because those are the two orthogonalizers implemented in the principal arms: MGS is what the baseline integrator runs (Algorithm 2), while CholeskyQR2 is the block orthogonalizer adopted by the CA arm. CholeskyQR2 orthogonalizes an entire block with two Gram-formation phases regardless of block width. Its attraction here is its communication count and measured cost; the reach of the available sufficient bound is assessed separately. TSQR reaches a comparable synchronization count through a reduction tree at higher implementation complexity [10, 19]; it is measured later as a robustness comparison.

The *reduction ledger* (Table 8) is operation counting: for each Krylov dimension m , it records the number of global reductions in one Arnoldi cycle. MGS pays $1 + m(m + 3)/2$; a block method with $q_{\max}$ columns pays $1 + 2\lceil m/q_{\max} \rceil$. Using the instrument’s guard-admitted legacy value $s_{\max} = 7$, hence $q_{\max} = 8$, gives an **11–16× reduction-count cut** across the measured $m = 8$ –11 window. The count is exact conditional on using that width, while numerical acceptance is evaluated separately through the factorization and solution diagnostics. Empirically, the real operator’s Krylov dimension is small ($m = 6$ –8 over eigensolvable grids $n = 10$ –20, and 7.6 to 11.3 over $n = 31$ –89), so a cycle occupies two or three such blocks in the reported runs.

Table 8.: Conditional reduction ledger at $q_{\max} = 8$ block columns (the archived instrument records the equivalent maximum degree $s_{\max} = 7$).

m	MGS reductions	s -step + CholQR2	cut
5	21	3	$7.0\times$
8	45	3	$15.0\times$
9	55	5	$11.0\times$
11	78	5	$15.6\times$
12	91	5	$18.2\times$

8.5. EMPIRICAL LOG-SENSITIVITY TO NON-NORMALITY

The spectral prediction of block conditioning assumes normality, and the real operator is not normal (mixed-derivative cross terms plus convection). The prediction error is measured as $|\log_{10} \kappa_{\text{meas}} - \log_{10} \kappa_{\text{pred}}|$, in decades. This is the natural scale for the implemented guard: each extra power in $[\mathbf{v}, \mathbf{A}\mathbf{v}, \dots]$ raises the measured condition number by a median of 1.05 decades over the sampled family (inter-quartile range 0.95 to 1.12). Approximately 1.2 decades therefore correspond to one change in the guard-admitted polynomial degree for these samples. The eigenvector condition number supplies a worst-case multiplicative distortion to the normal-operator factorization; it does not turn the guard into a CholeskyQR2 theorem.

The measured data sits below that worst-case envelope (Figure 7). Descriptive least-squares slopes of prediction error against $\log_{10} \kappa(X)$ are 0.069 for the sampled constant-coefficient mechanisms and 0.250 for the sampled variable-coefficient advection arm. These slopes summarize the observed design; sample-count uncertainty, residual diagnostics, and an out-of-sample check were not archived, so they are not presented as laws and are not extrapolated over additional decades. Their useful conclusion is narrower: the variable-coefficient sample is more sensitive than the constant-coefficient sample. Both fits use the canonical $\kappa(X)$ axis of Section 8.6.

The real Black–Scholes operator is shown as a test point and excluded from both fits. Its prediction error is 0.19–0.85 decades at measured $\kappa(X)$ up to 2.3×10^3 , less than the approximately 1.2-decade change associated with one guard step in this sample. The statement is confined to the measured grids; no extrapolation beyond that range is used.

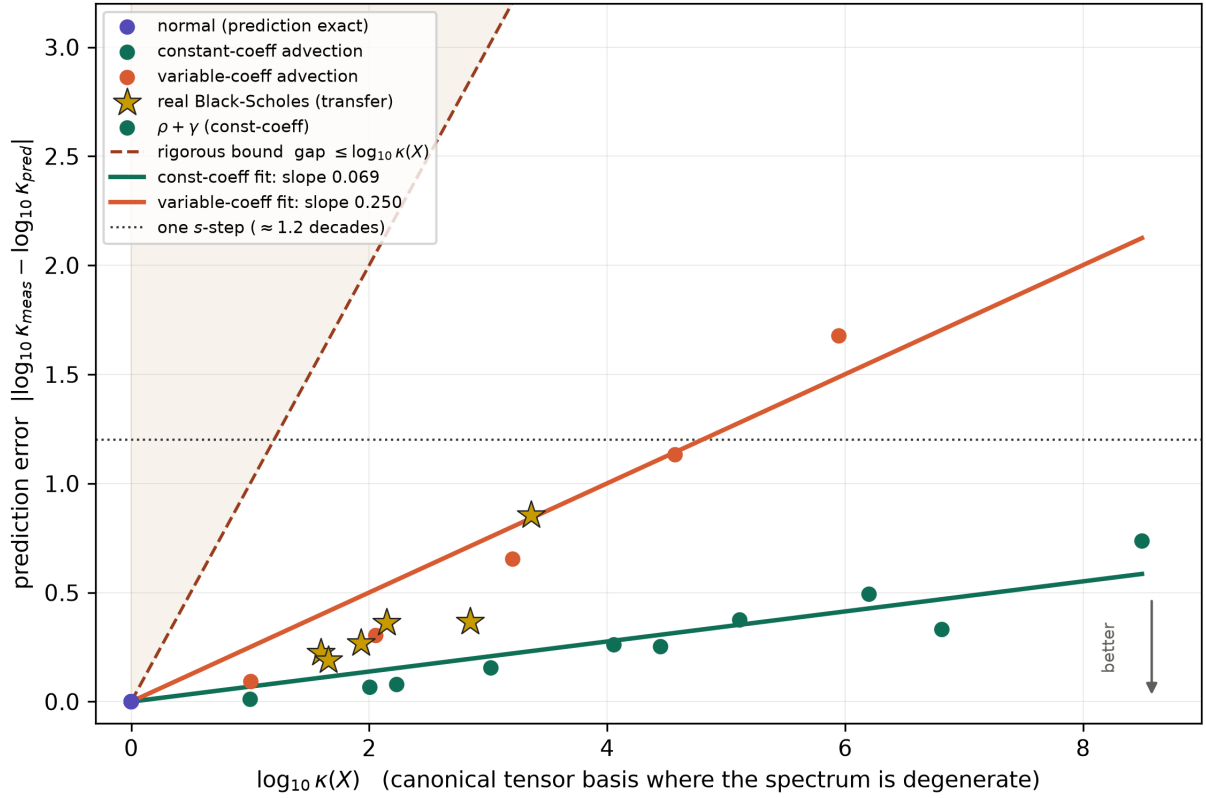


Figure 7.: Empirical log-sensitivity: prediction error in decades vs. $\log_{10} \kappa(X)$. The identity line is the worst-case eigenvector-conditioning envelope; separate descriptive fits are shown for the sampled constant- and variable-coefficient families. The real Black-Scholes operator is a test point excluded from both fits.

Operator by operator, what the spectral prediction needs is normality; definiteness and separability are conveniences (Table 9).

Table 9.: Which structural properties the spectral prediction of $s_{\max}$ actually needs.

Operator	Normal?	Definite?	Separable?	Spectral prediction
Laplacian	yes	PSD	yes	exact (closed form)
shift μ	yes	indefinite	yes	exact ($\kappa(X) = 1$)
correlation ρ	yes	PSD	no	exact (dense eigensolve)
constant γ advection	no	—	yes	survives (< 1 step)
$\rho + \gamma$	no	—	no	survives (< 1 step)
variable γ advection	no	—	no	must be measured (> 1 step)

The rows are cumulative modifications of one backbone. The Laplacian is the pure-diffusion Kronecker sum of Section 8, whose eigenvalues are the closed-form tridiagonal-Toeplitz values of Section 8.7 summed across axes. The shift row adds $\mu \mathbf{I}$, which translates every eigenvalue and leaves the eigenvectors untouched. The correlation row adds the cross term $c(\mathbf{D} \otimes \mathbf{D})$; the advection rows add the skew-symmetric first difference, with strength γ either constant or ramped with position. The three column properties are standard [25, 15]: an operator is *normal* when $\mathbf{A}\mathbf{A}^\top = \mathbf{A}^\top\mathbf{A}$, equivalently when it has an orthonormal eigenbasis; *definite* when it is symmetric with eigenvalues on one side of zero; and *separable* when it factors as a Kronecker sum. Normality is what the Vandermonde factorization of Section 8.3 needs to hold exactly. Separability decides whether its eigenvalues come from a closed form or a dense eigensolve. In the last column, *exact* means the predicted and measured guard degrees

agree as integers; *survives* means they differ by less than one degree; and *must be measured* means the discrepancy is larger. These labels compare the spectral predictor with the operational guard; they do not add a theorem-level stability claim.

8.6. CONDITIONING GROWTH WITH ASSET DIMENSION

The remaining conditioning question is how $\kappa(X)$ grows with the number of assets d . This axis matters because the application grows the asset count, not only the grid. A refinement study swept the grid at fixed $d = 3$ and found $\kappa(X)$ approximately flat. The mesh Péclet number, $\text{Pe}_h = \gamma \Delta x / (2\nu)$, explains this behavior for the synthetic family: with $\log \kappa(X) \sim dn_1\gamma$ and $\gamma \sim C/n_1$, the grid factor cancels, leaving d as the growth axis [20].

The growth is linear in $\log_{10} \kappa(X)$ against d , with a slope known in closed form. The synthetic operator applies its cross term to exactly one axis pair at every d , so it factors as a Kronecker sum $\mathbf{A}_d = \mathbf{B}_{01} \oplus \mathbf{T} \oplus \dots \oplus \mathbf{T}$, where $\mathbf{B}_{01}$ is the coupled two-axis block and $\mathbf{T}$ the one-dimensional factor. The eigenvector matrix of a Kronecker sum is the Kronecker product of the factors' eigenvector matrices, and the singular values of a Kronecker product are all pairwise products of the factors' singular values [15]—so condition numbers multiply:

$$\kappa(X) = \kappa(X_{01}) \kappa(X_1)^{d-2} \quad \implies \quad \log_{10} \kappa(X) = 0.40499 d + 0.01422. \quad (8.6.1)$$

“Closed form” means precisely this: the slope $0.404995 = \log_{10} \kappa(X_1)$ is computed from one $n_1 \times n_1$ eigensolve of the 1D factor, not fitted to data along d . The observed $d = 2 \rightarrow 3$ increment is 0.40500, and the formula reproduces both degeneracy-free points to 3×10^{-7} .

The dense $\kappa(X)$ returned by a particular eigensolve must not be fitted along d once repeated eigenvalues appear. Inside a repeated eigenspace the eigenvector basis is not unique, so the condition number of the assembled eigenvector matrix becomes basis-dependent even though invariant subspaces and spectral projectors remain meaningful [15]. Here, axes $2, \dots, d-1$ carry no cross term, so interchanging them is an exact symmetry of $\mathbf{A}$ and creates repeated eigenvalues. Across eight symmetric permutations, the dense value moves by a factor of 3.84 at $d = 4$ and 7.76 at $d = 5$. The study therefore reports a canonical Kronecker-factor value (`kappa_X_struct`) alongside the basis-dependent dense output (Table 10, Figure 8).

Table 10.: Canonical vs. dense $\kappa(X)$ along the asset dimension.

d	N	canonical $\kappa(X)$	dense $\kappa(X)$	spectrum
2	16	6.671	6.671	simple (agree to 3×10^{-7})
3	64	16.95	16.95	simple (agree to 3×10^{-7})
4	256	43.07	109.5	repeated: dense basis-dependent
5	1024	109.4	5.2×10^3	repeated: dense basis-dependent
6	4096	278.1	3.7×10^4	repeated: dense basis-dependent

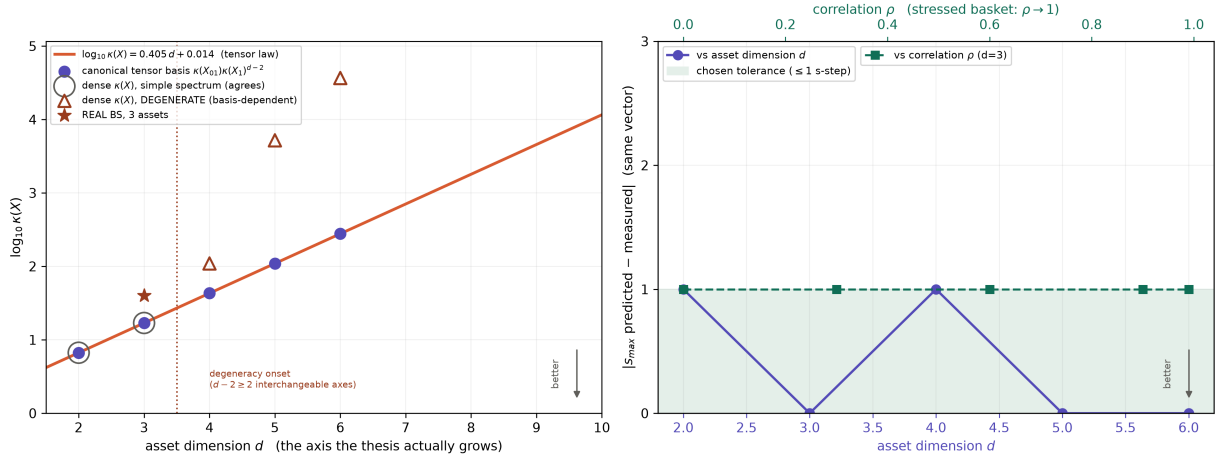


Figure 8.: Conditioning growth with asset dimension: canonical $\kappa(X)$ grows geometrically in d with an analytic slope; the dense eigenvector-basis value becomes basis-dependent when repeated eigenspaces appear.

The same repeated-eigenspace issue appears at one point of the correlation sweep ($n_1 = 8$, $d = 3$), and the canonical construction supplies a consistent comparison. For every $\rho > 0$ in this sweep the cross term couples one axis pair, the spectrum is simple, and the dense and canonical values agree. At $\rho = 0$ the operator becomes a pure Kronecker sum with all three axes interchangeable; only 89 of its 512 eigenvalues are distinct, so the dense eigenvector-basis value is basis-dependent. On the canonical construction (Table 11), the sampled values increase from 688 at $\rho = 0$ to 3405 at $\rho = 0.99$.

Table 11.: The correlation arm ($n_1 = 8$, $d = 3$). At $\rho > 0$ the dense value matches the canonical one; at $\rho = 0$ the repeated eigenspaces make the dense eigenvector-basis value basis-dependent.

ρ	0	0.3	0.6	0.9	0.99
canonical $\kappa(X)$	688.0	851.8	1566.9	2277.9	3405.2
dense $\kappa(X)$	<i>11350</i>	851.8	1566.9	2277.9	3405.2

Within this synthetic design, changing ρ over the sampled admissible range changes the canonical conditioning by about the same amount as adding 1.7 assets, while the dependence on d continues geometrically. A width selected at low asset count should therefore not be carried to higher d without new conditioning and accuracy measurements. The identity's scope is the synthetic instrument: the real basket operator couples all three asset pairs, so the one-pair tensor factorization does not extend to it.

8.7. THE COMPUTABILITY GAP

The spectral conditioning prediction is evaluated by dense eigensolve, which caps that control at $N \leq 32,768$, while the performance runs use $N \sim 10^5$. A natural bridge suggests itself: each one-dimensional factor is Toeplitz, and the eigenvalues of a tridiagonal Toeplitz matrix are known in closed form, $\lambda_k = a + 2\sqrt{bc} \cos(k\pi/(n+1))$ [15]. An analytic spectrum could therefore replace the eigensolve for a separable operator.

The bridge fails for this operator, and the obstruction is identifiable. The cross term factors exactly as $c(\mathbf{D} \otimes \mathbf{D})$ with $\mathbf{D} = \text{tridiag}(-1, 0, +1)$ (verified to machine zero), and a diagonal similarity $\text{diag}(z^j)$ symmetrizes $\text{tridiag}(b, a, c)$ iff $z^2 = c/b$. The two terms demand incompatible z on the same axis:

$z^2 = (1 - \gamma)/(1 + \gamma)$ for advection–diffusion against $z^2 = -1$ for the cross factor. Numerically, the sine basis leaves 3.7% off-diagonal mass with the cross term against 2.8×10^{-14} without, no single z helps, and the obstruction persists at $\gamma = 0$ —so it is the Dirichlet boundary condition, not the advection. What survives is the tensor identity (8.6.1), which gives $\kappa(X)$ at any d from a single $n_1^2 \times n_1^2$ eigensolve, with no cap—for the *synthetic operator*. The gap remains open for the real Black–Scholes operator, whose all-pairs correlation breaks the Kronecker factorization.

8.8. THE HORIZONTAL MECHANISM

Cutting the reduction count 11–16 $\times$ leaves Gram–Schmidt wall-clock unchanged. Two independent measurements explain why, and they agree:

1. **The primitive was never the bottleneck.** Replacing the naive barrier-plus- $O(P)$ -scan reduction with a proper binary tree—a $1.7\times$ improvement in the bare all-reduce latency at $P = 24$ ($2.91\ \mu\text{s}$ down to $1.70\ \mu\text{s}$)—leaves Gram–Schmidt wall-clock unchanged to within 5% at every core count. The OpenMP runtime’s barrier is already a tree, and the barrier itself dominates the scan.
2. **Gram–Schmidt is vector-bound, not reduction-bound.** MGS re-reads column $\mathbf{V}_i$ once for every $j \geq i$, so $m(m+1)/2$ column reads at $8N$ bytes each. That quadratic is a *bandwidth* term, and it is what the roll-off in the scaling study is actually made of.

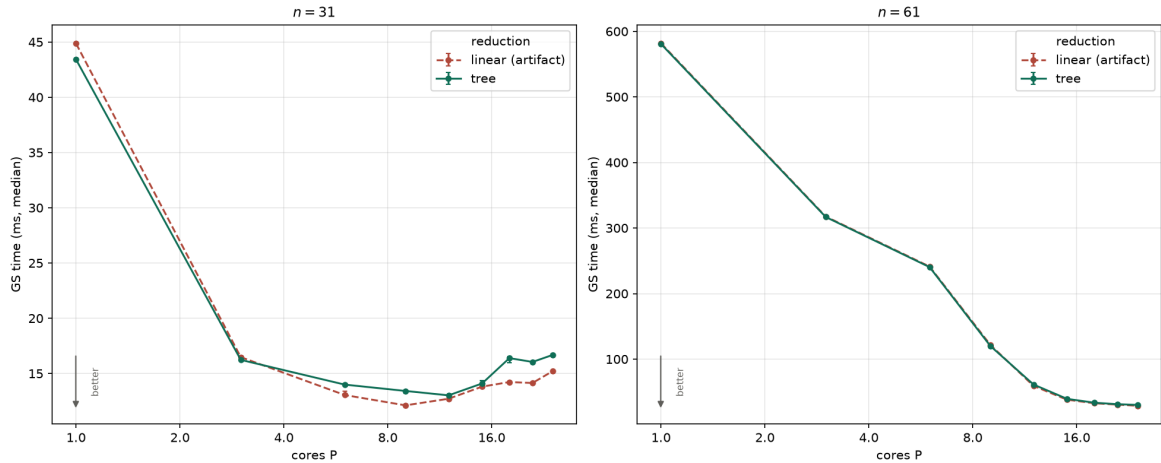


Figure 9.: Gram–Schmidt wall-clock under the calibrated tree reduction vs. the $O(P)$ redundant scan it replaced ($n = 61$, arm B; one contaminated $P = 15$ launch, inflated across all kernels, is excluded). A $1.7\times$ faster primitive leaves the kernel unchanged within 5%.

The map places the production-resolution case at $R_h = 0.080 \ll \theta_h = 1$, so the horizontal mechanism does not pass the nominal opportunity gate on this CPU. The measured null result is consistent with that placement; it does not establish that the performance crossover is exactly at one. The thesis therefore leads with the timed result, not the reduction count.

No measured or calibrated configuration on puffin enters the upper-right opportunity region. It is a single-NUMA socket, so the reduction tree crosses no boundary more expensive than a CCX; under the present model, reaching $R_h = 1$ while $R_v > 1$ would require a reduction cost roughly $25\times$ higher than the calibrated range. This is a topology-specific model statement, not a structural impossibility for all single-socket implementations.

8.9. THE VERTICAL MECHANISM

The horizontal verdict came almost for free, but the vertical one requires its own experiment. The roofline of Section 6 says the spilled SpMV has $4\text{--}8\times$ of recoverable traffic, yet (6.0.1) is only a ceiling: it prices the bytes tiling would save, not the redundant halo arithmetic tiling spends to save them, so whether a real matrix-powers kernel comes out ahead is a question only a timed run can answer. A timed sweep therefore runs a cache-blocked matrix-powers kernel against the untiled baseline across grids that carry R_v from 0.035 to 86. The measurement is not close to the modeled ceiling (Table 12, Figure 10).

Table 12.: The vertical sweep: measured speedup of the tiled matrix-powers kernel against the modeled traffic ratio.

Pattern	Tile level	R_v (1 core)	Measured speedup	Modeled ratio
banded	L3	0.035–2.69	0.72–1.31 $\times$	4–8 $\times$
banded	L2	1.12–86.1	0.37–0.89 $\times$	4–8 $\times$
scattered	L3	0.035–2.69	1.00 $\times$ (falls back to naive)	4 $\times$

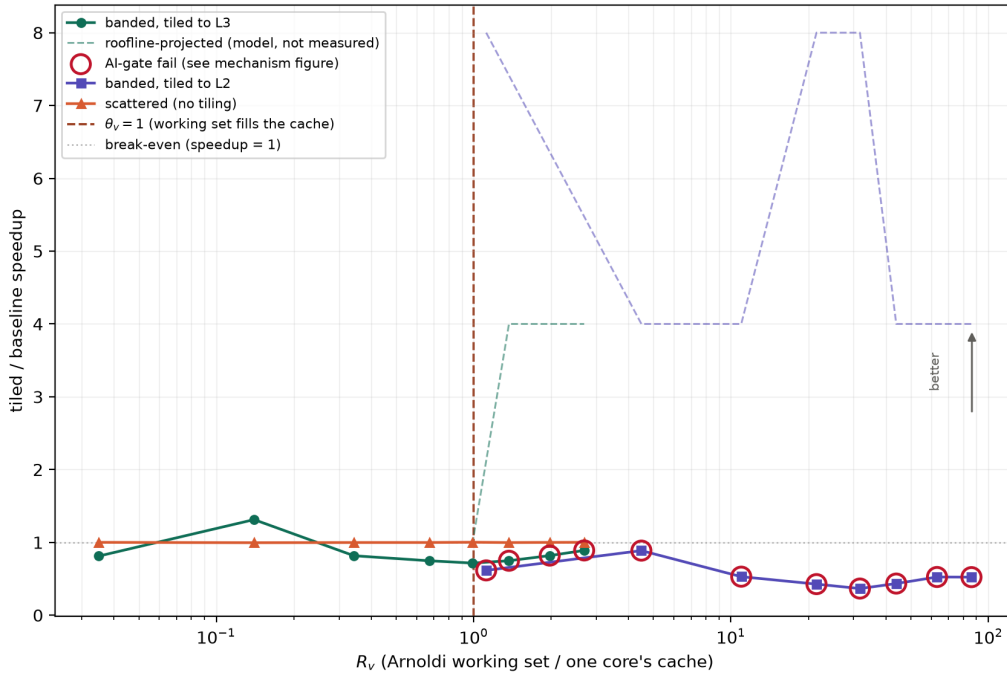


Figure 10.: The vertical sweep: measured speedup of cache-blocked matrix powers across R_v . The projected curve is an explicit model claim, drawn dashed, never presented as data.

The shortfall is a **kernel ceiling, not a refuted boundary**: the tiled arm never reached a bandwidth-bound regime, so the experiment never got to test θ_v . A diagnostic kernel that skips the halo entirely (wrong basis, throughput only) separates the two tile levels cleanly (Figure 11). At L2 (small panels, fat halo), removing the halo recovers throughput toward baseline (1.17 rising to 2.13 against 2.70 at $n_1 = 400$): cache-blocking works, and the shortfall is redundant halo arithmetic traded for DRAM traffic at a loss. At L3 (large panels, thin halo), removing the halo barely moves throughput (2.06 to 2.11 against 2.76): neither arm is bandwidth-bound, and the cap is insensitive to cache residency—most consistent with gather latency on the indirect column access.

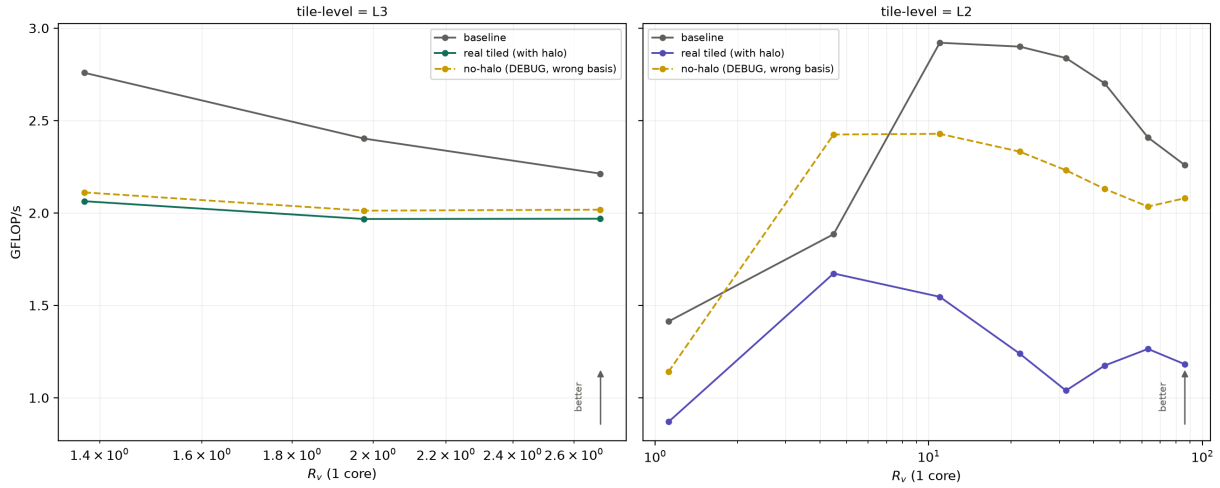


Figure 11.: The no-halo diagnostic. L2 tiling recovers toward baseline without the halo (the halo trade is the shortfall); L3 tiling barely moves (the cap is insensitive to cache residency).

The scattered arm delimits the contiguous geometric tiling implemented here. The permutation destroys the band/halo layout required by this kernel, so this implementation falls back to the naive form and measures $1.00\times$ by construction. The similarity transform leaves the spectrum and working set fixed, isolating sensitivity to storage order. It does not make sparse matrix-powers blocking mathematically impossible: graph reorderings and level-based methods such as RACE address more general sparse patterns and are outside the present implementation.

Two honest limitations. Nothing here is a *counted* DRAM measurement, because puffin’s uncore counters need root, so modeled bytes are reconciled against the roofline instead—weaker evidence, labeled as such on every figure. And the projected curve on Figure 10 is a model claim, drawn dashed.

8.10. BLOCK WIDTH

In the CPU regime instrument, the block is pinned at the largest degree admitted by the implemented conditioning guard, which hides a question. A larger block means fewer reductions and more powers per operator stream, but two independent ceilings pull its width down:

- **numerical (the implemented guard):** $\kappa([\mathbf{v}, \mathbf{A}\mathbf{v}, \dots, \mathbf{A}^{q-1}\mathbf{v}])$ grows rapidly with q ; past the heuristic $u^{-1/2}$ line the current implementation narrows the block. This is a software policy, not the dimension-dependent sufficient condition in (3.2.1).
- **capacity (the ghost):** the halo grows with recurrence depth, so a wider block shrinks the panel until none fits and the tiling collapses.

Which arrives first is a joint property of operator and machine, and on puffin it depends on the tiling level (Table 13).

Table 13.: The two observed limits on block degree. The L3 ghost limit is quoted as $\geq$ because a panel still fit at every swept degree up to 24; the numerical column records the operational $u^{-1/2}$ guard rather than the dimension-dependent sufficient limit.

Tile level	n_1	Heuristic degree limit	Ghost degree limit	First observed limit
L3	420	8	≥ 24	heuristic guard
L3	480, 560	6	≥ 24	heuristic guard
L2	420	8	3	capacity
L2	480	6	3	capacity
L2	560	6	2	capacity

The distinction is actionable because the two limits respond to different changes. The numerical guard is reached according to how fast the block condition number grows per power, which depends on the polynomial basis. Newton and Chebyshev bases are standard approaches to moderating that growth [3, 19, 7]. Where the implemented guard is encountered first, a better-scaled basis may admit a wider empirical block; where ghost capacity is encountered first, only the geometry or operator bandwidth moves the limit. Neither conclusion supplies a theorem for the distributed CholeskyQR2 factorization.

8.11. PLACING THE REAL OPERATOR ON THE MAP

The measured real-operator points anchor the map; a modeled continuation reports how the two coordinate definitions change on larger grids. The same algebra reproduces the measured (R_v, R_h) coordinates to better than 0.001%, which checks the implementation of the formulas but does not validate a performance extrapolation (Table 14).

Table 14.: The real operator’s modeled trajectory under grid refinement. Bisecting on n puts the nominal $R_v = 1$ opportunity crossing at $n \approx 73$.

n	N	m	R_v	R_h	Source
10	1,000	6	0.0019	13.53	measured
20	8,000	8	0.0175	1.87	measured
25	15,625	8	0.036	0.904	predicted
61	226,981	12	0.582	0.080	predicted
74	405,224	13	1.063	0.047	predicted (first above θ_v)
120	1,728,000	15	4.74	0.0025	predicted

In summary, the early $R_h > 1$ readings occur only on the small grids in the dense-eigensolve range. At production resolution the model places the operator in the vertical-only *opportunity* region: working-set pressure motivates testing matrix-powers tiling, while the horizontal ratio does not motivate s -step on this CPU. That placement does not assert that tiling pays; the timed vertical sweep shows why geometry can prevent it. The upper-right region is not measured on puffin: the model estimates that reaching it would require a reduction roughly 25 $\times$ more expensive than the calibrated CPU range.

Costs of that order can arise when a collective crosses NUMA, socket, device, or node boundaries, which is why s -step and pipelined methods are studied on distributed systems [8, 10, 19, 7, 4, 14]. The syngel cluster—two nodes with two Tesla V100s per node, PCIe within a node and a network fabric between nodes—provides several such topology rungs. The next section recomputes the opportunity

coordinates for that distinct memory system and compares them with measurements; it does not assume that the CPU unit lines are performance crossovers on the GPU.

9. RECOMPUTING THE MAP FOR A GPU

9.1. A CROSS-ARCHITECTURE TRANSFER TEST

Because R_v and R_h are dimensionless, their definitions can be evaluated for another operator–machine pair. That is a weaker and more defensible transfer claim than asserting a universal crossover at one. Puffin supplies one CPU cache hierarchy; a GPU changes both cache ownership and reduction mechanisms. This section therefore asks whether the same opportunity coordinates remain interpretable after their machine-dependent terms are recomputed, and whether the measured kernel behavior is consistent with that placement. A disagreement may expose a missing term rather than invalidate the use of dimensionless coordinates altogether.

Two GPU classes provide an informative architectural contrast (Table 15). The RTX 3090 and V100 have the same nominal 6 MB L2 capacity and similar measured streaming bandwidth in this study, while their FP64 peaks differ by about $11\times$. The contrast does not vary only the compute roof: the devices also differ in SM organization, caches, clocks, memory technology, ECC behavior, scheduling, and launch behavior. It can reveal a roofline distinction for the measured kernels, but it is not a controlled causal experiment.

Table 15.: Two GPU architectures measured on idle devices. Memory bandwidth is achieved (STREAM); the FP64 ridge is peak FP64 over achieved bandwidth. Similar nominal L2 capacity and measured bandwidth make the contrast useful, but architectural differences beyond FP64 remain uncontrolled.

	L2	Memory BW	FP64 peak	FP64 ridge
RTX 3090 (consumer, puffin)	6 MB	821 GB/s	0.570 TFLOP/s	0.69 FLOP/B
Tesla V100 (datacenter, synge)	6 MB	818 GB/s	6.375 TFLOP/s	7.79 FLOP/B

The contrast exposes a precondition that the two-coordinate map otherwise carries implicitly. Both axes describe communication opportunities and presuppose that the relevant kernel is on the memory side of its roofline. A kernel that is compute-bound lies outside that interpretation. Placement is therefore preceded by the roofline check $AI \cdot BW_{\text{achieved}} < \text{FLOP}/s_{\text{peak}}$ in the relevant precision. For the measured kernels, this check distinguishes a memory-bound V100 Gram kernel from a compute-bound RTX 3090 Gram kernel. The result scopes the mechanism; it does not isolate FP64 throughput from the other architectural differences.

9.2. WHAT TRANSFERS, AND WHAT BREAKS

The synthetic operator is machine-independent. So are the tensor identity in (8.6.1) and the conditional reduction ledger. The CholeskyQR2 guard remains an operational policy on both machines, with its sufficient theory assessed separately, and the unit lines $\theta_v = \theta_h = 1$ remain nominal equality lines rather than portable performance thresholds. The machine-dependent coordinate terms must be recomputed. Three changes are structural rather than constant substitutions:

- **The denominator of R_v .** On the CPU each engaged CCX brings its own L3 slice, so aggregate last-level cache scales with the team. A GPU’s L2 is a single fixed block shared by every streaming multiprocessor: engaging more of the device adds zero cache, so within one device R_v stops being a function of the team size at all. This decouples the two axes. On the CPU both coordinates shared the parallelism knob ($R_v \sim 1/P$, $R_h \sim P \log P$), which is why $R_v \cdot R_h$ collapsed to a P -independent constant (~ 0.04 on puffin) in the calibrated model, excluding the upper-right region over the measured CPU configurations. On a GPU that cancellation has no structural reason to survive, so whether the product remains an invariant must be re-derived from the chosen definition of team size, not assumed.
- **The numerator of R_h .** The CPU model has two reduction tiers and one saturation constant (8.1.1). A GPU reduction crosses at least five qualitatively different tiers (Table 16), and the grid tier carries kernel-launch latency, which has no CPU analogue—a fixed cost per reduction independent of tree depth. Two parameters cannot express five tiers: the machine model needs a tier vector, and the reduction-cost predictor needs to walk it. Three reachable rungs with distinct costs (on-device, intra-node across PCIe, inter-node across the fabric) place the same problem at three different R_h values. This traverses the modeled horizontal coordinate; the performance crossover remains a separate measured outcome.
- **The reuse model.** The CPU scatter knob costs a factor of a few in cache-line reuse; on a GPU the same permutation defeats warp coalescing and can serialize a 32-thread gather into 32 transactions. The scatter axis becomes more discriminating, but the reuse model must be recalibrated from scratch.

Table 16.: The reduction-tier ladder for a multi-node GPU cluster with a PCIe intra-node interconnect. Every tier cost is a machine constant to be calibrated, never assumed.

Tier	Mechanism	Scope	Cost class
Warp	register shuffle	on-die	near-free, no memory traffic
Block	shared memory + barrier	on-die	cheap, on-chip
Grid	atomics or a second launch	on-die	expensive; carries launch latency
Device	PCIe + cross-socket link	intra-node	expensive; a genuine cross-socket hop
Node	network fabric	inter-node	dominant

9.3. THE DECOUPLED INVARIANT

On the CPU model the product $R_v R_h$ is approximately independent of the core count P because $R_v \sim 1/P$ and $R_h \sim P \log P$ move in opposite directions. On a GPU, take P to be the number of engaged streaming multiprocessors. R_v ’s denominator is one shared L2, while R_h uses a device-wide bandwidth roof after saturation, so the coordinates are modeled as decoupled from P : grid size moves R_v and reduction tier moves R_h . In the memory-bound regime indicated by the roofline check, the modeled cycle time reduces to bytes over bandwidth and the product takes the form

$$R_v \cdot R_h = g(m) R(m) \Lambda, \quad \Lambda = \frac{\tau \text{ BW}}{C}, \quad (9.3.1)$$

in which $g(m)$ (a byte ratio) and $R(m)$ (the MGS reduction count) are the machine-independent method terms, and Λ —the only place the machine enters—is dimensionless and reads as how many cache-fulls of data the memory system delivers during one reduction. Two consequences follow at

once. First, the upper-right window (both $R_v \geq 1$ and $R_h \geq 1$ at a footprint under the 16 GB device memory) is non-empty exactly when this product exceeds 1, and its width in N is the product: the two questions of whether a corner exists and how wide it is collapse to one. Second, the 16 GB ceiling never binds—it is out by two orders of magnitude at every reachable tier—so the constraint that closes the window from above is R_h , the reduction cost, not capacity. Setting the product to 1 and solving for the reduction latency gives the recomputation’s single sharpest prediction: a reduction costing more than $\tau^* \approx 0.9 \mu\text{s}$ carries the production operator into the corner.

The modeled equality occurs at $\tau^* \approx 0.9 \mu\text{s}$. The V100 reduction ladder is calibrated with isolated collectives at the stated payloads and no application work (Table 17); it places the same operator at five R_h values. The nominal upper-right opportunity region begins between the block and grid rungs, inside one device. This is a model placement, not a measured solver crossover at τ^* . Two features of the ladder are notable. Node-to-node cost is only $1.9\times$ the device-to-device cost on this topology, while the largest step is the grid reduction’s second kernel launch. Participant count, topology, and software launch costs therefore cannot be collapsed to a single universal node-tier constant.

Table 17.: The calibrated V100 reduction ladder. Each rung is an isolated collective at its stated scope and τ is cumulative. The modeled upper-right opportunity region ($R_v R_h > 1$) begins between the block and grid rungs; this is not by itself a performance crossover.

Rung	Scope	τ (cumulative)	Corner
warp (register shuffle)	on-die	$0.42 \mu\text{s}$	closed
block (shared + barrier)	on-die	$0.79 \mu\text{s}$	closed
grid (second kernel launch)	on-die	$5.54 \mu\text{s}$	open
device (PCIe + cross-socket)	intra-node	$11.46 \mu\text{s}$	open
node (inter-node fabric)	inter-node	$22.09 \mu\text{s}$	open

9.4. REACHING THE CORNER, AND THE GATE THAT DISCRIMINATES

At production resolution ($n_1 = 61$, $m = 12$, legacy instrument degree $s = 8$), the recomputed coordinates place the operator at $R_v = 7.2$, $R_h = 1.7$ for the selected reduction rungs on both cards, inside the upper-right *opportunity* region. This demonstrates that the dimensionless definitions can be evaluated on the GPU hierarchy after their constants are recomputed. It does not show that the speedup boundary is unchanged or that both mechanisms pay.

The coordinates presuppose a communication-bound kernel, so the roofline check is essential (Table 18). SpMV (arithmetic intensity 0.135 FLOP/B) and MGS (0.375) are on the memory side of both measured roofs. The tall-skinny Gram matrix introduced by the block method has intensity that grows with block width and, at the tested production width, is memory-bound on the V100 but compute-bound on the RTX 3090. The implemented $u^{-1/2}$ guard admits this width; numerical acceptance uses the empirical diagnostics described in Section 3.2, since the dimension-dependent sufficient condition does not cover it. The roofline result explains why the two-coordinate communication interpretation applies to the V100 Gram kernel and not to the RTX 3090 Gram kernel; it does not predict the end-to-end speedup from the coordinates alone.

Table 18.: The roofline gate at production resolution, per kernel. The baseline kernels are on-map on both cards; the s -step Gram block is memory-bound only on the datacenter card.

Kernel (AI, FLOP/B)	V100 (ridge 7.79)	3090 (ridge 0.69)
SpMV (0.135)	on map	on map
MGS (0.375)	on map	on map
s -step Gram (~ 1.1)	memory-bound ($3.9\times$)	compute-bound

9.5. ROOFLINE-BOUND KERNELS AND THE s -STEP CROSSOVER

Reading the roofline distinction on the working kernel required a formulation suited to very tall, narrow blocks. In the benchmark harness, cuBLAS `dsyrk` and `dtrsm` attained low utilization for these shapes. Two specialized kernels were therefore written: a split-K Gram that parallelizes the long dimension and reduces partials [12], and a per-row triangular solve exploiting independent row solves. They agree numerically with the cuBLAS results and, within this harness and timing scope, are reported as roughly $1900\times$ and $27\times$ faster respectively (Table 19). These ratios are shape- and harness-specific; they are not a general comparison with current cuBLAS, cuBLASLt, or CUTLASS, and should be interpreted together with setup, synchronization, and launch amortization in the archived measurement procedure.

Table 19.: The two hand-written tall-skinny kernels at the production block width. Both reach the roofline; the Gram classifications differ across the two measured architectures.

Kernel	vs. cuBLAS	V100	3090
split-K Gram	$\sim 1900\times$	85% DRAM roof (memory-bound)	71% FP64 peak (compute-bound)
per-row trsm	$27\times$	95% DRAM roof (memory-bound)	94% roof (at the knee)

With roofline-bound kernels the two-card contrast becomes a hardware measurement rather than a prediction. Profiled with Nsight Compute at $s = 8$, $n = 2 \times 10^6$, the *same* Gram kernel runs at 86% of DRAM throughput and 16% of compute on the V100 (memory-bound, FP64 idle) and at 38% of DRAM against 94% of compute on the 3090 (compute-bound). The triangular solve, whose intensity (~ 0.56) sits just below the 3090’s ridge, is memory-bound on the V100 and at the compute/memory knee on the 3090, so the Gram is the sole unambiguous carrier of the negative arm—exactly the kernel the roofline gate named. On consumer silicon the CA treatment’s own orthogonalization kernel is compute-bound while the baseline it replaces is memory-bound on both cards.

The reduction saving itself was then timed. Against MGS at matched $m = 12$, the block method—with observed orthogonality $\|\mathbf{I} - \mathbf{Q}^T \mathbf{Q}\| \sim 10^{-15}$ —trades MGS’s 91 reductions for 5. On a single device the specialized block kernels beat MGS at every tested grid on both cards. Across two GPUs, every dot or block Gram becomes an NCCL all-reduce over the device link; the measured $\sim 11.5\mu\text{s}$ latency agrees with the independently calibrated rung. At that topology the five-reduction block is faster than the 91-reduction MGS block over the tested range. This is an orthogonalizer experiment, not an end-to-end price comparison: it shows behavior consistent with the horizontal opportunity coordinate after calibration, while the performance crossover remains an observed, topology-specific result.

10. THE DISTRIBUTED CA EXPONENTIAL INTEGRATOR

Section 9 recomputed the opportunity coordinates for a GPU memory system and measured the two mechanisms on isolated kernels. That prices an orthogonalizer rather than a solve. This section evaluates a complete communication-avoiding exponential integrator for the pricing problem of Section 2, distributed over spatial subdomains on up to four V100s across two nodes. The question is whether the kernel-level saving survives convergence control, the implemented conditioning guard, polynomial boundary forcing, and comparison with an independent semi-discrete referee.

10.1. THE SOLVER

The grid is cut into *slabs* along the slowest axis x_3 , one slab per GPU, so each device owns a contiguous range of grid planes together with the whole Krylov basis for its own rows. A slab decomposition is the natural choice here because the operator’s bandwidth is $O(N_1 N_2)$ (Section 2): cutting along x_3 makes every off-slab dependency a plane of the neighbor’s data, rather than the surface of a three-dimensional box.

That dependency is the *halo* (or ghost region): the planes a device must import before it can apply the operator to its own boundary rows. The halo is where the vertical mechanism reappears at network scale. One application of the stencil reaches one plane beyond the slab; a matrix-powers block of width s applies the operator $s - 1$ times, so it reaches $s - 1$ planes out. Exchanging those $s - 1$ planes in a single *deep* message replaces $s - 1$ shallow exchanges. The bytes moved are the same—this is important, and Section 10.2 measures it—but the message count falls from $s - 1$ to one, and on a latency-dominated fabric messages are what is paid for.

The rest follows the structure of Section 3.2. The operator is applied matrix-free: the 19-point stencil is evaluated from grid coordinates, with no assembled matrix and therefore no index traffic, and a thread block stages an interior tile plus its ghost layers in shared memory and runs the recurrence there. Each basis block is orthogonalized by CholeskyQR2. The device evaluates the operational $\kappa_2(\mathbf{Z}) \leq u^{-1/2}$ guard and narrows a block that exceeds it. As established in Section 3.2, this policy targets practical normal-equations breakdown, while the dimension-dependent sufficient condition and distributed Gram rounding are treated separately [27, 13]. The block Gram matrix and projection are formed locally and combined with NCCL all-reduces; the halo travels as NCCL send/receive pairs [28]. Because $m \leq 24$, the projected $\mathbf{H}_m$ is at most 24×24 , so the small exponential is replicated on every device.

Correctness is checked against an instrument that shares no stencil code with the production path. An independent referee assembles the $N + 3$ basket operator $\tilde{\mathbf{A}}$ (or the unaugmented rainbow operator) as a sparse matrix, applies it with the vendor library, and evaluates the exponential action by the Al-Mohy–Higham Taylor method [2]. Reported arms pass the implemented convergence and conditioning guards, preserve the forcing tail or rainbow boundary state, and agree with an arm-matched single-GPU and independent semi-discrete reference within the stated tolerances. These checks provide direct evidence for the tested numerical path; theorem-level CholeskyQR2 coverage and a total financial-error budget remain separate validation questions. Device contention is queried before timing, and a contended run withholds its timing distribution. Adaptive decisions are compared across ranks before collectives so disagreement aborts rather than deadlocks.

10.2. RECURRENCE DEPTH, AND WHAT A FAIR CONTROL COSTS

A block of width s consumes basis columns $0, \dots, s-1$. It therefore needs $s-1$ recurrence steps and a halo $s-1$ planes deep. The delivered implementation requested s of each and discarded the extra column—a defect worth recording in full, because of what it does to the *control* rather than to the treatment.

At $s=1$ the block build then performs one operator application and one halo exchange whose only output is a copy of its input. A width-one CA arm is the natural non-CA baseline, so the reported saving was measured against a control doing twice the operator work and twice the halo traffic of a standard Arnoldi iteration. Correcting it was expected to *reduce* the measured CA speedup, since the correction removes more work from $s=1$ than from $s=4$.

The correction is a separately named arm rather than an overwrite. The delivered implementation is frozen as **as-measured**; the corrected recurrence runs beside it as **exact-depth**, in the same binary, on the same devices, in the same job. The comparison between the two is itself the result, and it is reported in the direction it came out. Table 20 gives the outcome in the natural invariant: halo planes moved per Krylov basis vector.

Table 20.: Halo planes moved per Krylov basis vector, $n=97$ on four V100s. The predicted column was recorded before the corrected run; the measured column is read from the solver’s own exchange-depth histogram.

Arm	s	Exchanges/step	Planes per vector	Predicted
as-measured	1	22.87	2.000	2
as-measured	4	6.39	1.250	1.25
exact-depth	1	11.43	1.000	1
exact-depth	4	6.38	0.997	1

Two results follow. First, the corrected $s=1$ arm applies the operator once and exchanges one halo per basis vector, which is exactly what modified-Gram–Schmidt Arnoldi does; it differs from a standard method only in running CholeskyQR2 on width-one blocks. It is therefore an admissible non-CA control, and every speedup below is quoted against it. Second, under the correction both widths move the same halo volume per basis vector, so **s -step buys message count, not message volume**—the textbook prediction, now measured rather than asserted. The corrected $s=4$ arm in fact moves *more* halo bytes per step than $s=1$ (5,628 against 5,041 KiB), because its fixed depth-one transition exchange is amortized over four vectors while its deep exchange shrank only from four planes to three. The correction changes communication and not the recorded semi-discrete result: state values and endpoint defect indicators agree with the frozen record to eight digits on every arm.

10.3. STRONG SCALING

The workload is $n=61$ (226,981 unknowns), 100 steps, tolerance 10^{-8} , $m_{\max}=24$, monomial basis, CholeskyQR2. Every value is a seven-repeat median on idle, NVML-gated devices (Table 21, Figure 12).

Table 21.: Strong scaling at $n = 61$: cycle time in ms/step, seven-repeat medians, and the CA speedup $t(s=1)/t(s=4)$. The frozen **as-measured** arm and the corrected **exact-depth** arm were run on the same devices in the same allocation.

Topology	Basket			Rainbow		
	$s=1$	$s=4$	CA	$s=1$	$s=4$	CA
<i>as-measured</i>						
One V100	2.969	2.857	1.04×	2.964	2.840	1.04×
Two V100s, one node	3.506	2.817	1.24×	3.409	2.756	1.24×
Two V100s, two nodes	4.726	3.587	1.32×	4.752	3.506	1.36×
Four V100s, two nodes	4.778	2.827	1.69×	4.792	2.685	1.79×
<i>exact-depth</i>						
One V100	2.418	2.675	0.90 ×	2.399	2.665	0.90 ×
Two V100s, one node	3.042	2.225	1.37×	3.005	2.203	1.36×
Two V100s, two nodes	3.659	2.870	1.27×	3.725	2.820	1.32×
Four V100s, two nodes	4.499	2.428	1.85 ×	4.398	2.401	1.83 ×

The corrected column carries two findings, and they point in opposite directions.

The apparent single-GPU crossover does not survive. At one participant the corrected $s = 4$ arm is 0.90×—slower than its control. The frozen 1.04× arose because the control was charged for a polynomial power it did not consume. A single GPU has no inter-device collectives or halo exchange to avoid, so this point supplies a local-work control: the wider block has no measured end-to-end benefit in that setting. It does not, by itself, prove that every difference at four GPUs is caused by communication, because participant count also changes occupancy, launch behavior, and subdomain size.

The distributed advantage is topology dependent. Across the full ladder the corrected CA ratios are 0.90, 1.37, 1.27, and 1.85× for Basket, and 0.90, 1.36, 1.32, and 1.83× for Rainbow. The two-node/two-GPU point interrupts a monotone trend, so the data support neither a participant-count scaling law nor extrapolation beyond the measured systems. The corrected four-GPU value nevertheless exceeds the frozen one. The pre-run forecast expected the opposite because it assumed a constant cost per operator application. In the measured implementation, reducing the block build from four recurrence steps to three also shrinks the shared tile from 2,304 doubles per 128 interior points to 1,400, whereas reducing it from one step to none removes a 360-double tile. Section 10.5 measures the sensitivity to this tile geometry directly.

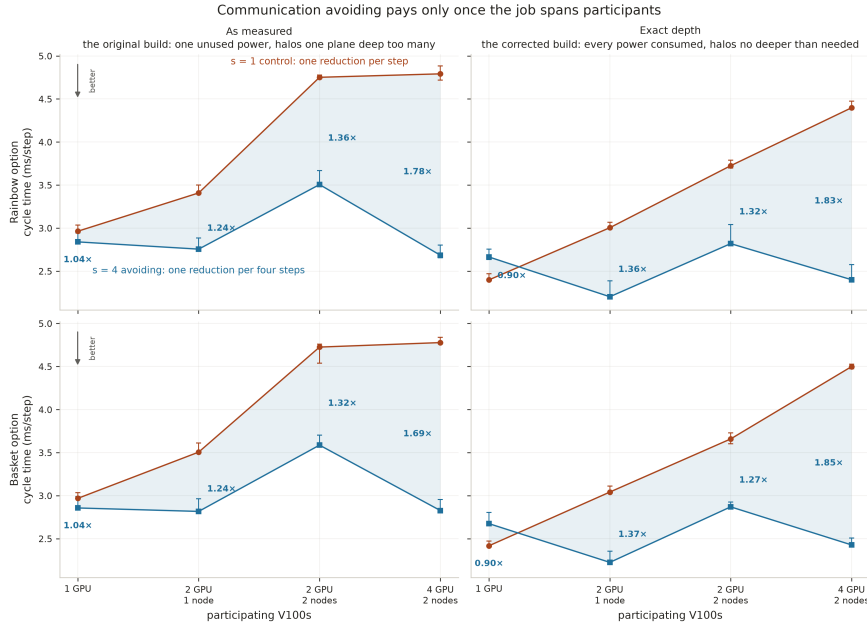


Figure 12.: Strong scaling at $n = 61$ on Synge V100s, both arms and both payoffs. Marks are seven-repeat medians with the observed range. The corrected single-GPU point falls below $1\times$; the corrected four-GPU point rises above the frozen record.

Absolute scaling at this size is poor. Four V100s at $s = 4$ beat one V100 at the same width by $1.11\times$ —about 28% parallel efficiency—and merely tie the best single-GPU configuration, which is $s = 1$ (2.401 against 2.399 ms/step). At $n = 61$ each of four slabs owns roughly 57,000 rows, too little to saturate a V100. The $1.83\times$ algorithm-relative comparison and the $1.11\times$ same-width absolute scaling result answer different questions; neither should be substituted for the other.

10.4. THE COMMUNICATION ACCOUNT

Interpreting the measured difference requires a cost model that is not fitted to the solver. The model is built from independent communication-only microbenchmarks: bare all-reduces and halo exchanges at the payloads the integrator uses, timed on idle devices for each measured participant count (Table 22).

Table 22.: Participant calibration on idle Synge V100s across two nodes: seven-repeat median latency of a bare NCCL all-reduce at the payloads the solver uses. Doubling the participant count is a real cost that a single node-tier constant cannot express.

Operation	Payload	2 participants	4 participants	Ratio
Norm	8 B	20.06 μ s	29.20 μ s	1.46 $\times$
Gram, $s=1$	8 B	20.94 μ s	28.26 μ s	1.35 $\times$
Projection, $s=1$	200 B	22.32 μ s	31.35 μ s	1.40 $\times$
Gram, $s=4$	128 B	21.91 μ s	31.16 μ s	1.42 $\times$
Projection, $s=4$	800 B	23.32 μ s	38.31 μ s	1.64 $\times$
64 MiB all-reduce	64 MiB	9.51 ms	12.51 ms	1.32 $\times$

Every collective and every halo exchange the solver performs is counted from its own transcript, priced at these calibrated latencies—halos at the depth each arm actually exchanged—and subtracted from the measured cycle. The remainder is local work, so no solver-time coefficient is fitted (Figure 13). On

the four-GPU runs this account assigns 31–35% of the $s = 1$ cycle and 11–14% of the $s = 4$ cycle to communication. Participant count and payload therefore account for much of the observed algorithm-relative difference, but the subsequent non-additivity check prevents this decomposition from being read as a causal proof.

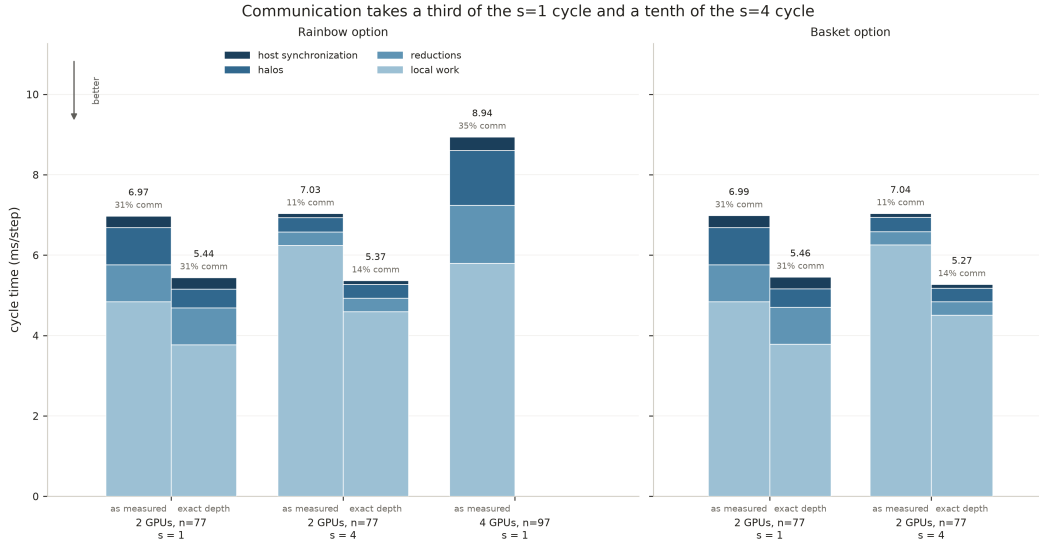


Figure 13.: Cycle-time decomposition on Synge V100s across two nodes. Reductions and halos are transcript counts priced by the participant calibration; local work is the remainder, so no term is fitted. Bars carry the total cycle time and the share of it spent communicating.

Agreement is close but not exact, and the residual is itself a finding: the communication floor is **not additive**. Summing reductions, deep exchanges, and host round-trips over-counts, because those costs overlap device work rather than serializing behind it. The clearest case is the host-synchronization term. The conditioning diagnostic is read back to the host to decide whether to narrow the block; this is implementation overhead, so it is reported separately. At $n = 61$ on four GPUs the corrected $s = 1$ arm performs 35.1 such round-trips per step and stalls 2.04ms; the $s = 4$ arm performs 10.6 and stalls 1.64ms—fewer synchronizations, each draining more queued work. Because the stall is the host waiting for work the device is already doing, it is an upper bound on recoverable overhead and is never summed into the account. Naming it explicitly keeps an implementation artifact from being read as a property of the method.

10.5. THE VERTICAL COORDINATE, MEASURED

The CPU study left the vertical boundary untested rather than refuted: the tiled kernel never reached a bandwidth-bound regime (Section 8.9), and its DRAM traffic was modeled rather than counted. On the GPU both limitations lift. Hardware counters are available without privilege, so 44 sweep points were captured with the profiler on an idle device—four widths, four grids, both payoffs, and four tile geometries—with measured DRAM traffic beside modeled demand (Table 23, Figure 14).

Table 23.: The matrix-powers kernel against both roofs at $n = 97$ (Rainbow, production $8 \times 4 \times 4$ tile, idle V100, counter-measured DRAM traffic). Redundancy is the ratio of shared-tile reads to distinct interior points. No width comes within reach of either roof.

s	Ghost redundancy	DRAM roof	L2 roof	Verdict
1	$2.81 \times$	54.9%	3.8%	latency-bound
2	$6.00 \times$	21.1%	2.3%	latency-bound
3	$10.94 \times$	8.0%	1.6%	latency-bound
4	$18.00 \times$	2.1%	1.2%	latency-bound

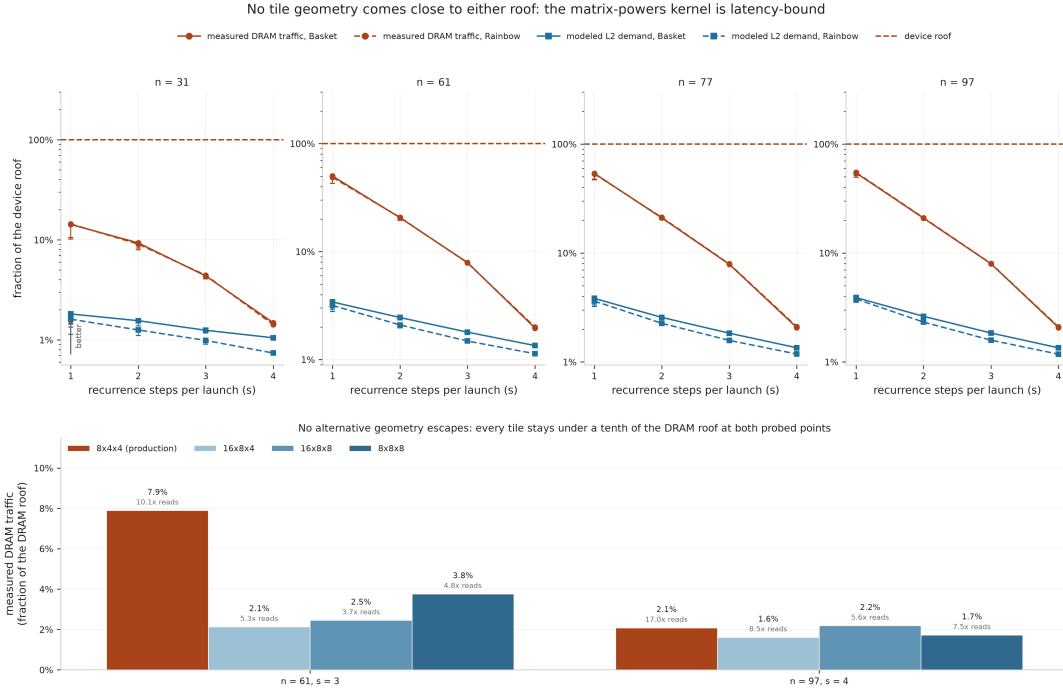


Figure 14.: Matrix-powers traffic as a fraction of the device roofs on an idle V100, seven repeats per point. Top: the production tile swept over recurrence steps at each grid, with counter-measured DRAM traffic against the DRAM roof and modeled tiled demand against the L2 roof. Bottom: the alternative tile geometries at the two points where they were probed, labeled with effective in-domain read redundancy.

The verdict required changing the classifier, and the change is recorded here rather than applied silently. The original rule labeled a width *L2-absorbed* when measured DRAM traffic fell well below modeled tiled demand. That is a true statement about *where the traffic went*, and not a statement about *what limits the kernel*: a kernel running at 1.2% of the L2 roof is not L2-limited. The two questions are now separated. Traffic attribution is reported as the ratio of measured to modeled bytes, which runs 3.3, 2.0, 1.1, 0.40 as s goes 1 to 4—the model under-counts at narrow tiles, where a tile row straddles several memory sectors, and over-counts at wide ones, where the overlap between neighboring blocks is L2-resident. Limiter attribution tests roofs only, and returns *latency-bound* at every one of the 44 points: the largest DRAM fraction anywhere is 0.55 against a 0.75 threshold, and the largest L2 fraction is 0.04. An occupancy test was part of the original cascade and was removed because, within this controlled sweep, lower-occupancy configurations sometimes ran faster.

What does bind is the tile. Table 24 sweeps four geometries at the two production points, and a better tile is worth more than the s -step crossover itself.

Table 24.: Tile geometry sweep on an idle V100, seven-repeat medians. Redundancy is the effective in-domain read redundancy; speedup is against the production $8 \times 4 \times 4$ tile at the same point, given as the range over the two payoffs. Occupancy is quoted for $(n=61, s=3)$ and $(n=97, s=4)$ respectively.

Tile	Occupancy	$n = 61, s = 3$		$n = 97, s = 4$	
		Redund.	Speedup	Redund.	Speedup
$8 \times 4 \times 4$ (production)	0.250 / 0.125	10.07×	1.00×	17.03×	1.00×
$8 \times 8 \times 8$	0.125 / 0.0625	4.81×	1.33–1.39 ×	7.52×	0.96–1.02×
$16 \times 8 \times 8$	0.0625	3.69×	1.10–1.21×	5.62×	1.28–1.36 ×
$16 \times 8 \times 4$	0.0625	5.34×	0.95–1.04×	8.45×	0.98–1.04×

Neither candidate limiter survives the sweep. The winning geometry has *lower* occupancy than production in every case—quartering occupancy made the kernel faster. Ghost redundancy alone fails too: at $n = 61$ the lowest-redundancy tile ($16 \times 8 \times 8$) moves the least traffic and still loses to $8 \times 8 \times 8$; at $n = 97$ the lowest-traffic tile is 30% slower than the winner. The two effects trade against each other—a wider tile cuts redundant bytes but degrades access efficiency—and they balance at different points on different grids, which is why the best geometry is (n, s) -dependent and the production default is left unchanged. Inside the whole solver the better geometry is worth $1.08\times$ at $s = 1$ and $1.13\times$ at $s = 4$, against $1.33\text{--}1.39\times$ for the bare kernel; the dilution is the rest of the cycle.

This is the sharpest single statement the GPU work makes about the vertical mechanism, and it agrees in shape with the CPU result of Section 8.9 while identifying the cause the CPU study could only conjecture. On both machines the tiled matrix-powers kernel falls far short of the traffic ratio its roofline promises. On the CPU the shortfall was attributed to redundant halo arithmetic and to a residency-insensitive cap consistent with gather latency. On the GPU, with counted rather than modeled bytes, the answer is explicit: **the tile geometry, not the mechanism, caps the vertical payoff**. Adopting a better tile is a change to the production method and therefore a new research arm, not a correction—but the coordinate itself is now known to need a third term for tile redundancy before it can predict a vertical speedup rather than merely bound one.

10.6. THE CONDITIONING GUARD ON AN EVOLVING SOLVER BASIS

Section 8.3 evaluated the implemented $u^{-1/2}$ conditioning guard on a start-vector probe. The solver basis is a useful additional diagnostic because the starting vector changes from step to step. Table 25 reports the largest width admitted by that guard over an integration when four columns are requested. As noted in Section 3.2, this is an operational threshold: the dimension-dependent sufficient condition does not cover the measured production block, and the distributed all-reduce introduces rounding not represented by a local condition number alone. The evolving-basis measurements therefore complement rather than replace the direct orthogonality and referee checks.

Table 25.: Largest block width admitted by the implemented guard, and the largest block condition number observed over the integration, at a requested width of 4 (Rainbow, idle V100). The guard is $\kappa_2(\mathbf{B}) \leq u^{-1/2} \approx 9.5 \times 10^7$ and controls the fallback; numerical acceptance also uses the orthogonality and referee checks described in the text.

Basis	$n = 31$		$n = 61$	
	Admitted	$\max \kappa_2(\mathbf{B})$	Admitted	$\max \kappa_2(\mathbf{B})$
Monomial	4	6.0×10^6	4	6.0×10^6
Newton (real Leja)	3	8.5×10^7	3	9.3×10^7
Chebyshev	4	4.7×10^7	3	7.9×10^7

At these two grids and widths, the monomial blocks have the smallest observed condition numbers and are the only ones for which the guard admits all four requested columns. This is a local observation, not a general ordering of polynomial bases: Newton and Chebyshev recurrences are designed to control growth at larger degrees, and only widths up to four are tested here. The a-priori Gershgorin enclosure admits every requested column up to six, whereas the evolving-basis guard admits at most four. The enclosure is therefore too coarse to select a production width on its own.

The same blocks were also orthogonalized with TSQR and reorthogonalized block Gram–Schmidt. On identical $n = 61$, $m = 8$, $s = 4$ blocks, all three methods produced orthogonality losses between 3.5×10^{-14} and 6.5×10^{-14} , while CholeskyQR2 was fastest in this implementation (Table 26). These measurements compare observed orthogonality and time; they do not establish backward stability for the complete integrator.

Table 26.: Orthogonalization on identical blocks, $n = 61$, Rainbow, idle V100. The table reports observed orthogonality loss and cost; it does not establish full backward stability. Time is basis-plus- $\mathbf{H}$ assembly.

Method	Orthogonality loss	Time	Relative
CholeskyQR2	4.01×10^{-14}	2.248 ms	1.00×
TSQR [9]	3.52×10^{-14}	6.882 ms	3.06×
Block Gram–Schmidt, reorthogonalized	6.47×10^{-14}	2.778 ms	1.24×

10.7. WEAK SCALING AND THE CONVERGENCE STOP

Weak scaling holds the local volume near 227,000 rows per GPU, so the grid grows with the participant count: $n = 77$ on two GPUs and $n = 97$ on four, measured against a one-GPU run at $n = 61$. On the frozen arm, efficiency for the $s = 1$ control falls to 43.1% at two participants and 33.4% at four, while the $s = 4$ arm gives up less—40.9–41.1% at two and 39.1% at four. The corrected arm improves both; absolute times and the full efficiency table are in Section B, and the figure in Section A.

The experiment stops at two predeclared boundaries that are consistent with the same stiffness trend. The operator norm grows approximately as n^2 (Section 2): the independent sparse assembly gives 2.19×10^2 at $n = 31$ and 8.74×10^2 at $n = 61$, a ratio of 3.995 against $(61/31)^2 = 3.87$. With the step size held at $T/100$, inverting the Hochbruck–Lubich bound [18] gives order-of-magnitude Krylov dimensions (Table 27). The bound’s assumptions do not match this non-Hermitian operator, so it is used only as a diagnostic scale.

Table 27.: Implied Krylov dimension against $m_{\max} = 24$ at a fixed step of $T/100$. The bound is a Hermitian positive semi-definite theorem and this operator is neither, so these are order-of-magnitude figures—but they locate both observed stops.

n	$\ \mathbf{A}\ _1$	$h\ \mathbf{A}\ $	Implied m
61	874	8.7	15
77	1,392	13.9	19
97	2,209	22.1	24
381	34,078	340.8	94

At $n = 97$ the implied dimension is exactly $m_{\max}$, and the Basket arm there did reach $m_{\max} = 24$ on one step with an endpoint defect indicator of 1.118×10^{-8} , just above tolerance; it is drawn as stopped and never as a performance result. At $n = 381$ —the largest grid the allocation model admits on one V100 at $s = 4$, predicted before any memory was reserved and confirmed by allocating it (Table B.3)—the implied dimension is about 94, roughly four times what the solver permits. The recorded maximum endpoint defect indicator is 6.5×10^3 , so this arm is treated as unconverged rather than as a marginal failure. No parameter was changed after observing it. Making $n = 381$ converge by changing $m_{\max}$, the step count, or the tolerance would be a different experiment.

10.8. WHAT THE INTEGRATOR ADDS TO THE MAP

The map identifies two opportunities: expensive reductions can make fewer synchronization rounds valuable, and a large working set can leave traffic for matrix powers to reuse. The full solver produces results consistent with the first opportunity only after communication crosses device boundaries: s -step is worth $0.90\times$ on one V100 and $1.83\times$ on the measured four-GPU topology. The calibrated account assigns a substantially larger communication share to $s = 1$, but its non-additivity and the non-monotone topology ladder preclude a stronger causal or scaling claim. For the vertical opportunity, the matrix-powers kernel is latency-bound at every measured width and never approaches a roof; shared-tile geometry matters more than the nominal cache coordinate records. The coordinates are therefore useful opportunity descriptors, not performance guarantees.

11. FUTURE WORK

The study leaves several threads open, and the measurements above make five of them concrete.

MIXED PRECISION. Every kernel here runs in double precision. The matrix-powers recurrence and block Gram matrix are natural reduced-precision candidates, but the measurements do not determine a safe single-precision width. The values 5.9×10^2 at width two and 9.9×10^4 at width three exceed the heuristic single-precision threshold $u^{-1/2} = 4096$ at different widths, but $u^{-1/2}$ alone is not the applicable CholeskyQR2 theorem. A mixed-precision study should evaluate the dimension-dependent sufficient condition, the rounding from the distributed reduction, accumulation precision, and the final orthogonality and exponential-action defect together. A promising arm is a reduced-precision recurrence with higher-precision Gram accumulation and correction [17, 1]. Because the present matrix-powers kernel is latency-bound, reduced bytes should not be assumed to translate proportionally into reduced time.

A TILE-AWARE VERTICAL COORDINATE. R_v compares a working set against a cache and predicts, correctly, that there is traffic to recover. It does not predict whether a given kernel will recover it, and Table 24 shows the gap is large: a tile change is worth more than the mechanism the coordinate was built to price. The natural repair is a third factor multiplying the demand side of R_v —the effective in-domain read redundancy, computable a priori from the tile dimensions and the stencil radius. The 44-point sweep is already a validation set for such a term, since it varies geometry at fixed operator and fixed working set. Whether one scalar suffices, given that the winning geometry is (n, s) -dependent and that lower occupancy ran faster, is the open question.

MORE PARTICIPANTS, AND A SHORTER PATH FOR THE HALO. The four-GPU point establishes a result on that measured two-node topology, not a crossover law. The full ladder is non-monotone, and the participant calibration covers only one doubling at each payload (Table 22); larger allocations must therefore be measured rather than extrapolated. The inter-node halo also uses NCCL’s InfiniBand-verbs transport without GPUDirect RDMA, so enabling a direct path is a distinct experimental arm that could reduce the communication term both widths retain.

NUMERICAL DIRECTIONS. The operator is fixed across time steps and only the starting vector changes, so recycling or deflation could reduce the Krylov dimension m directly [23]. Rational Krylov remains an untested alternative, and the ADI splittings of Section 5 suggest possible preconditioners for its shifted solves. Block width could also adapt to the evolving-basis diagnostic, provided that the resulting policy is assessed through orthogonality and exponential-action accuracy together with the operational guard.

FINANCIAL VALIDATION AND COMPETITIVE BASELINES. The present experiments validate a semi-discrete exponential action; they do not give a total price or Greeks error budget. A pricing study should vary domain truncation, mesh width, time step, start-up smoothing, and interpolation, then compare price and sensitivity errors at equal accuracy. The reported production grids are odd and centre the spot exactly, so nearest-node extraction introduces no interpolation error for those base cases; off-centre spots and Greeks require an explicit interpolation study. Equal-accuracy comparisons should include optimized GPU ADI, modern low-synchronization Krylov variants, and QMC or other finance baselines appropriate to the payoff. This is the work needed before drawing a claim about practical pricing competitiveness.

SCOPE LEFT OPEN BY THE INSTRUMENT. The tensor identity (8.6.1) treats a single coupled axis pair, whereas the basket operator couples all $\binom{d}{2}$ pairs, so its conditioning growth with asset dimension is unmeasured beyond the dense-eigensolve range of Section 8.7. The distributed acceptance tests should also be restated as relative, arm-matched bounds where distinct reduction orders make overly tight absolute tolerances inappropriate. Finally, the V100 and 3090 results provide two architectural observations; broader hardware portability remains to be tested.

12. CONCLUSION

This thesis is a regime study of communication avoidance in a Krylov exponential integrator for two three-asset option PDE benchmarks. Its coordinates R_v and R_h are nominal opportunity measures: they identify where cache reuse or fewer reductions may matter, but their unit lines are not universal performance crossovers.

The CPU experiments separate the two mechanisms. Reducing the orthogonalization count by $11\text{--}16\times$ produced no measurable wall-clock gain in the tested single-socket setting, where reductions were not the limiting cost. The cache-blocked matrix-powers kernel also failed to realize the traffic opportunity predicted by its working-set ratio; halo arithmetic and implementation-specific latency limited the tested kernel before a clean cache crossover was reached. These null and shortfall results bound what the present implementations achieved rather than what every CA implementation could achieve.

The conditioning study is empirical by design. The $u^{-1/2}$ guard agreed with the observed admissible width in 20 of 21 recorded cases, and the log-sensitivity fits summarize the tested operator families. At the production dimension, the available CholeskyQR2 sufficient condition covers condition numbers only up to about 1.25×10^4 for $M = 226,984$ and four block columns, while the measured block reaches about 6×10^6 . The thesis therefore bases its numerical claims on the combined guard, orthogonality, factorization, and referee evidence within the tested range, without extrapolating the fitted relationships.

The GPU experiments add two architectural observations. The V100 and RTX 3090 differ in several material respects, so their contrasting roofline behavior is not a controlled FP64 experiment. On the measured V100 distributed solver, correcting recurrence depth removes a spurious single-GPU advantage: the $s = 4$ arm is $0.90\times$ the corrected $s = 1$ control on one GPU. On four V100s across two nodes it is $1.83\times$ faster than that control for Rainbow, while same-width strong scaling from one to four GPUs is only $1.11\times$, about 28% efficiency, and the best four-GPU time merely ties the best single-GPU time. The intermediate topology points are non-monotone, so these data do not establish a scaling law. Calibrated communication costs explain much of the algorithm-relative difference, but their overlap with device work prevents a fully additive causal account.

The principal conclusion is methodological. Communication-avoiding performance depends jointly on the operator, the numerical basis, kernel geometry, participant topology, and an honest equal-work control. The regime coordinates help organize those factors and the experiments show where their current form is incomplete, especially for tile redundancy and distributed synchronization. Because the validation here is semi-discrete, the thesis makes no claim about total option-price or Greeks accuracy, nor about competitiveness with current production pricing methods. Establishing those claims requires an equal-accuracy financial validation and broader hardware and algorithm baselines.

BIBLIOGRAPHY

- [1] Ahmad Abdelfattah, Hartwig Anzt, Erik G. Boman, Erin Carson, Terry Cojean, Jack Dongarra, Alyson Fox, Mark Gates, Nicholas J. Higham, Xiaoye S. Li, Jennifer Loe, Piotr Luszczek, Srikara Pranesh, Siva Rajamanickam, Tobias Ribizel, Barry F. Smith, Kasia Swirydowicz, Stephen Thomas, Stanimire Tomov, Yaohung M. Tsai, and Ulrike M. Yang. A survey of numerical linear algebra methods utilizing mixed-precision arithmetic. *International Journal of High Performance Computing Applications*, 35(4):344–369, 2021.
- [2] Awad H. Al-Mohy and Nicholas J. Higham. Computing the action of the matrix exponential, with an application to exponential integrators. *SIAM Journal on Scientific Computing*, 33(2):488–511, 2011.
- [3] Zhaojun Bai, Dan Hu, and Lothar Reichel. A Newton basis GMRES implementation. *IMA Journal of Numerical Analysis*, 14(4):563–581, 1994.
- [4] Grey Ballard, James Demmel, Olga Holtz, and Oded Schwartz. Minimizing communication in numerical linear algebra. *SIAM Journal on Matrix Analysis and Applications*, 32(3):866–901, 2011.
- [5] Mario Berljafa and Stefan Güttel. Parallelization of the rational Arnoldi algorithm. *SIAM Journal on Scientific Computing*, 39(5):S197–S221, 2017.
- [6] Fischer Black and Myron Scholes. The pricing of options and corporate liabilities. *Journal of Political Economy*, 81(3):637–654, 1973.
- [7] Erin Carson. *Communication-Avoiding Krylov Subspace Methods in Theory and Practice*. PhD thesis, University of California, Berkeley, 2015.
- [8] Anthony T. Chronopoulos and C. William Gear. s -step iterative methods for symmetric linear systems. *Journal of Computational and Applied Mathematics*, 25(2):153–168, 1989.
- [9] James Demmel, Laura Grigori, Mark Hoemmen, and Julien Langou. Communication-optimal parallel and sequential QR and LU factorizations. *SIAM Journal on Scientific Computing*, 34(1):A206–A239, 2012.
- [10] James Demmel, Mark Hoemmen, Marghoob Mohiyuddin, and Katherine Yelick. Avoiding communication in sparse matrix computations. Technical report, University of California, Berkeley, 2008. IEEE International Symposium on Parallel and Distributed Processing.
- [11] Jim Douglas and Henry H. Rachford. On the numerical solution of heat conduction problems in two and three space variables. *Transactions of the American Mathematical Society*, 82:421–439, 1956.
- [12] Dominik Ernst, Georg Hager, Jonas Thies, and Gerhard Wellein. Performance engineering for real and complex tall & skinny matrix multiplication kernels on GPUs. *International Journal of High Performance Computing Applications*, 35(1):5–19, 2021.

- [13] Takeshi Fukaya, Yuji Nakatsukasa, Yuka Yanagisawa, and Yusaku Yamamoto. CholeskyQR2: A simple and communication-avoiding algorithm for computing a tall-skinny QR factorization on a large-scale parallel system. In *Proceedings of the 5th Workshop on Latest Advances in Scalable Algorithms for Large-Scale Systems (ScalA '14)*, pages 31–38, 2014.
- [14] Pieter Ghysels and Wim Vanroose. Hiding global synchronization latency in the preconditioned conjugate gradient algorithm. *Parallel Computing*, 40(7):224–238, 2014.
- [15] Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. Johns Hopkins University Press, 4th edition, 2013.
- [16] Nicholas J. Higham. The scaling and squaring method for the matrix exponential revisited. *SIAM Journal on Matrix Analysis and Applications*, 26(4):1179–1193, 2005.
- [17] Nicholas J. Higham and Theo Mary. Mixed precision algorithms in numerical linear algebra. *Acta Numerica*, 31:347–414, 2022.
- [18] Marlis Hochbruck and Christian Lubich. On Krylov subspace approximations to the matrix exponential operator. *SIAM Journal on Numerical Analysis*, 34(5):1911–1925, 1997.
- [19] Mark Hoemmen. *Communication-Avoiding Krylov Subspace Methods*. PhD thesis, University of California, Berkeley, 2010.
- [20] Willem Hundsdorfer and Jan G. Verwer. *Numerical Solution of Time-Dependent Advection-Diffusion-Reaction Equations*, volume 33 of *Springer Series in Computational Mathematics*. Springer, 2003.
- [21] Karel J. in ’t Hout and Sven Foulon. ADI finite difference schemes for option pricing in the Heston model with correlation. *International Journal of Numerical Analysis and Modeling*, 7(2):303–320, 2010.
- [22] Marghoob Mohiyuddin, Mark Hoemmen, James Demmel, and Katherine Yelick. Minimizing communication in sparse matrix solvers. In *Proceedings of the Conference on High Performance Computing Networking, Storage and Analysis (SC '09)*, 2009.
- [23] Michael L. Parks, Eric de Sturler, Greg Mackey, Duane D. Johnson, and Spandan Maiti. Recycling Krylov subspaces for sequences of linear systems. *SIAM Journal on Scientific Computing*, 28(5):1651–1674, 2006.
- [24] Yousef Saad. Analysis of some Krylov subspace approximations to the matrix exponential operator. *SIAM Journal on Numerical Analysis*, 29(1):209–228, 1992.
- [25] Lloyd N. Trefethen and David Bau, III. *Numerical linear algebra*. SIAM, 6 1997.
- [26] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.
- [27] Yusaku Yamamoto, Yuji Nakatsukasa, Yuka Yanagisawa, and Takeshi Fukaya. Roundoff error analysis of the CholeskyQR2 algorithm. *Electronic Transactions on Numerical Analysis*, 44:306–326, 2015.
- [28] Ichitaro Yamazaki, Hartwig Anzt, Stanimire Tomov, Mark Hoemmen, and Jack Dongarra. Improving the performance of CA-GMRES on multicores with multiple GPUs. In *Proceedings of the*

IEEE 28th International Parallel and Distributed Processing Symposium (IPDPS), pages 382–391, 2014.

A. SUPPLEMENTARY FIGURES

The figures collected here support claims the body establishes by table or by argument. Each is reproducible from its own plotting script; every timing is a seven-repeat median on an idle, NVML-gated device, with the observed range as the error bar.

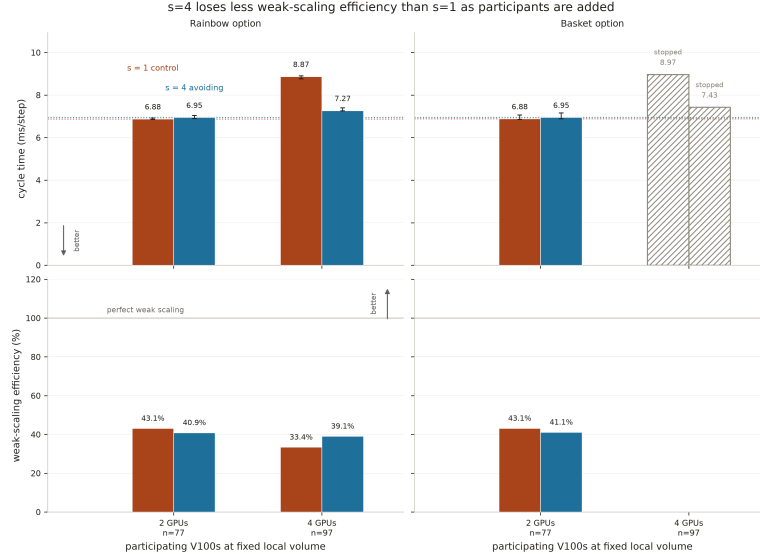


Figure A.1.: Weak scaling at a fixed local volume of $\approx 227,000$ rows per GPU (Section 10.7). Dotted rules mark perfectly flat weak scaling from the two-GPU value; efficiency is measured against a one-GPU run at the same local volume. The hatched bar left a Krylov cycle unconverged and is drawn as a stop, not as a performance result.

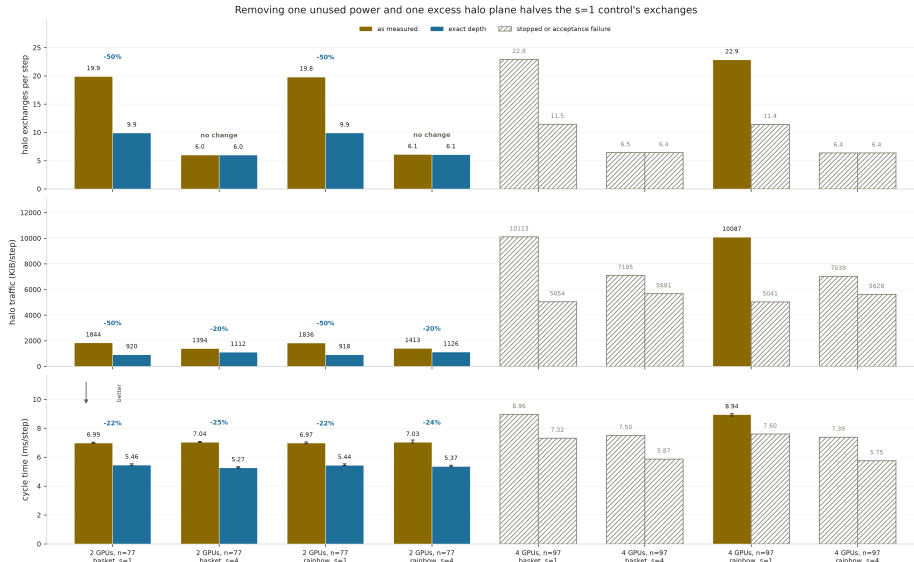


Figure A.2.: The recurrence-depth correction, arm against arm (Section 10.2). Both arms were attempted in one job on the same devices, so each pair is a controlled comparison rather than two runs from two allocations. Percentages are the change from **as-measured** to **exact-depth**; hollow hatched bars produced no result.

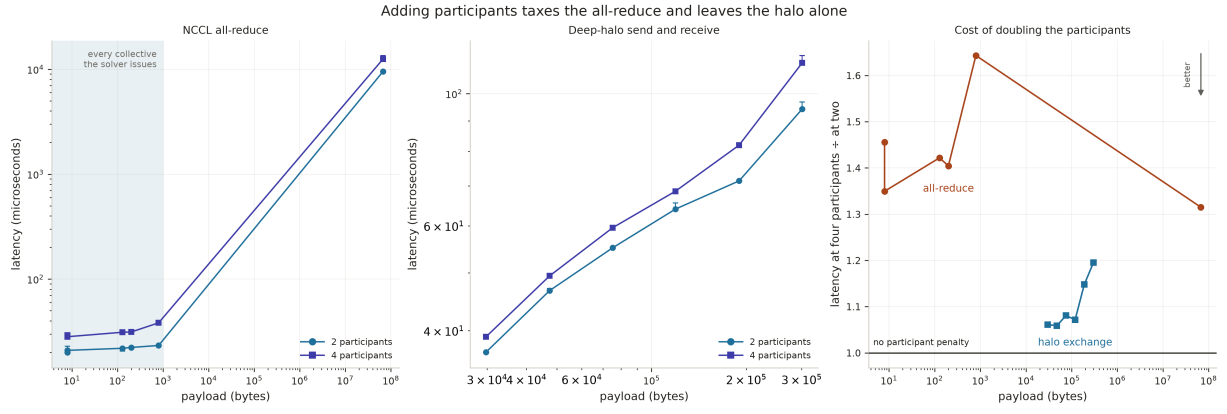


Figure A.3.: Collective and halo latency against payload on idle V100s across two nodes (Table 22, Table B.2). Both payload axes are logarithmic. The shaded band in the first panel is the payload range the integrator’s collectives occupy; the third panel is the ratio of the two, which is the participant-count term a single node-tier constant cannot express.

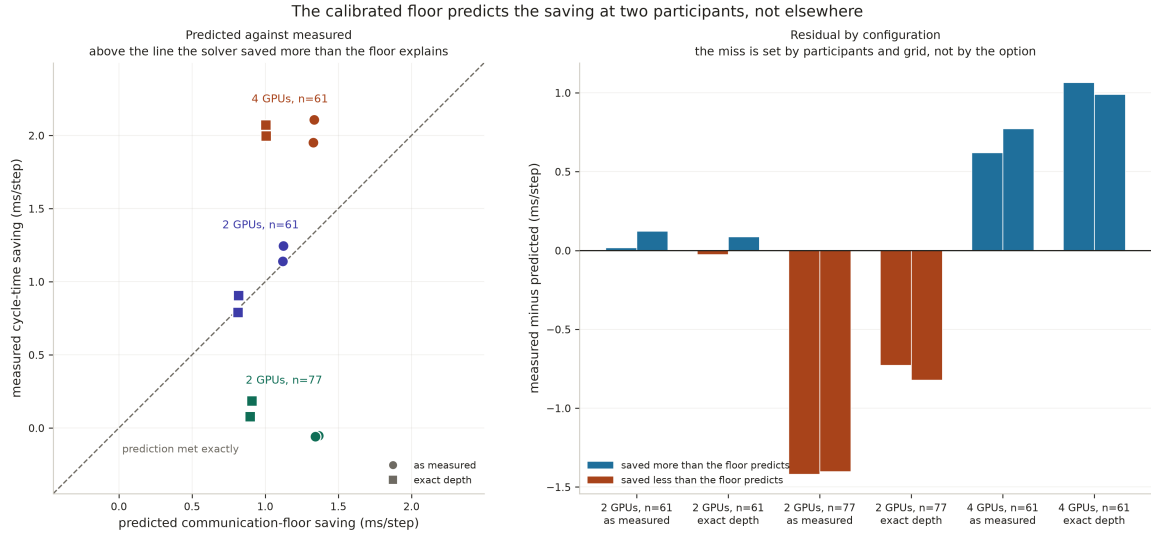


Figure A.4.: Calibrated communication-floor saving against measured cycle-time saving from $s = 1$ to $s = 4$ (Section 10.4). Both use the same time units, but the calibrated components overlap device work and cannot be added as a causal decomposition. The discrepancy records that non-additivity.

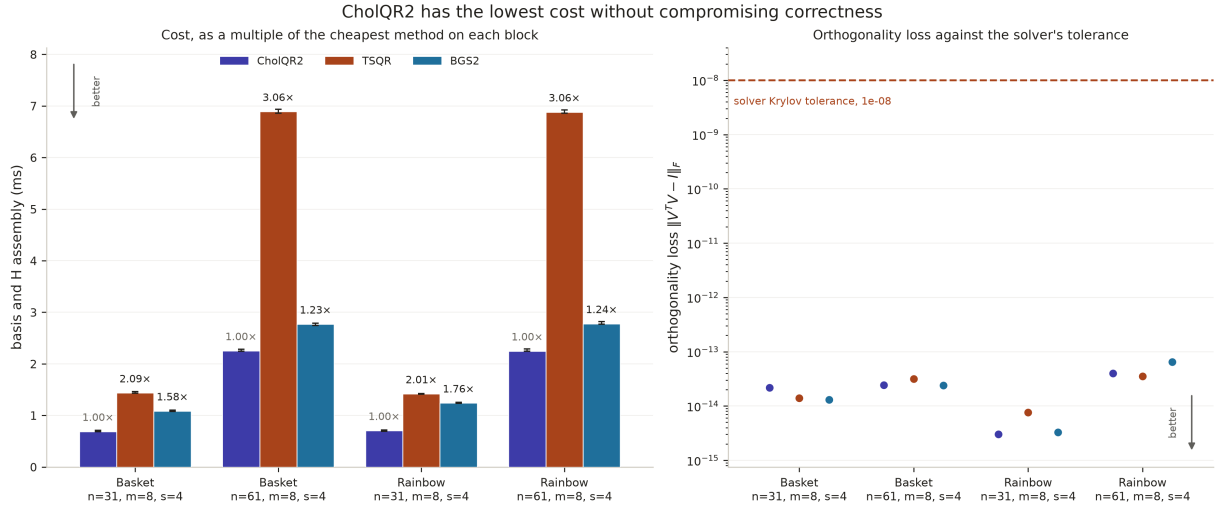


Figure A.5.: Orthogonalization cost and orthogonality loss on identical blocks (Table 26). Each group is one block: the same n , m , s , basis, and payoff across all three methods. The dashed rule is the solver's Krylov stopping tolerance, which is the scale on which an orthogonality loss either matters or does not.

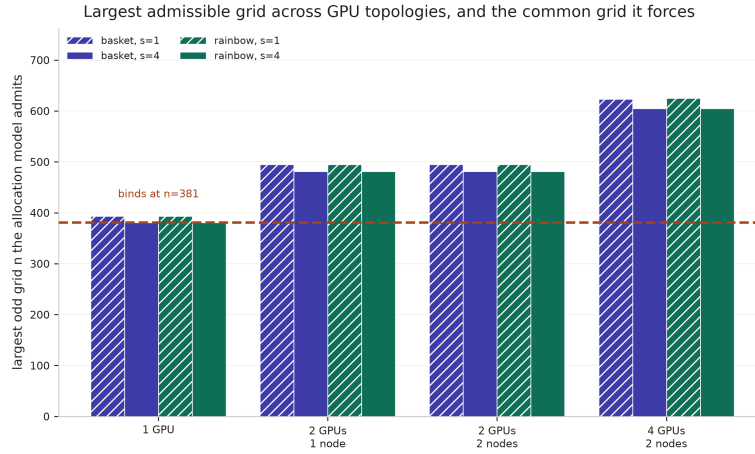


Figure A.6.: Largest odd grid the allocation model admits per topology, payoff, and block width (Table B.3). The dashed rule is the common grid selected for the sweep, the minimum over every arm. The prediction was made before any allocation, so the physical limit was a decision input rather than a job that dies minutes in.

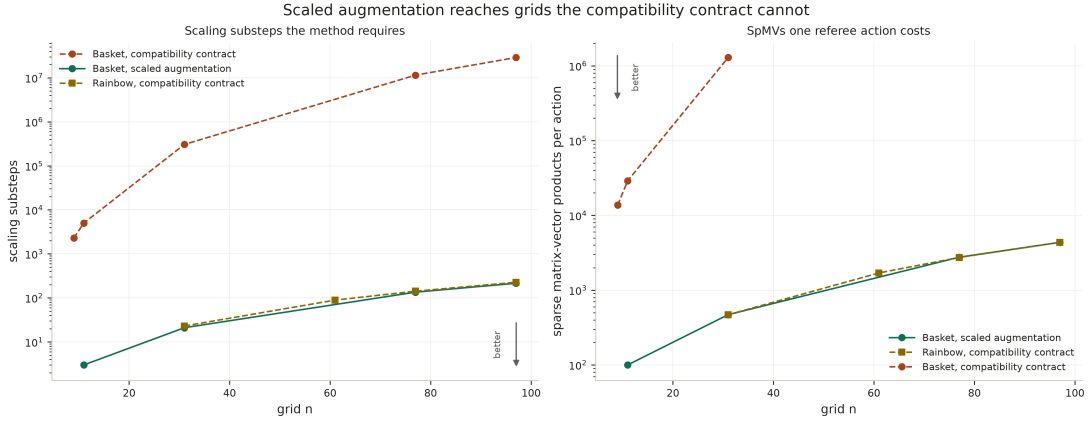


Figure A.7.: Cost of the independent referee against grid size (Section 10.1, Table B.4), for the fixed compatibility contract and the promoted scaled-augmentation method. Both vertical axes are logarithmic. The Basket compatibility contract was never run above $n = 31$, so its series stops where the evidence does.

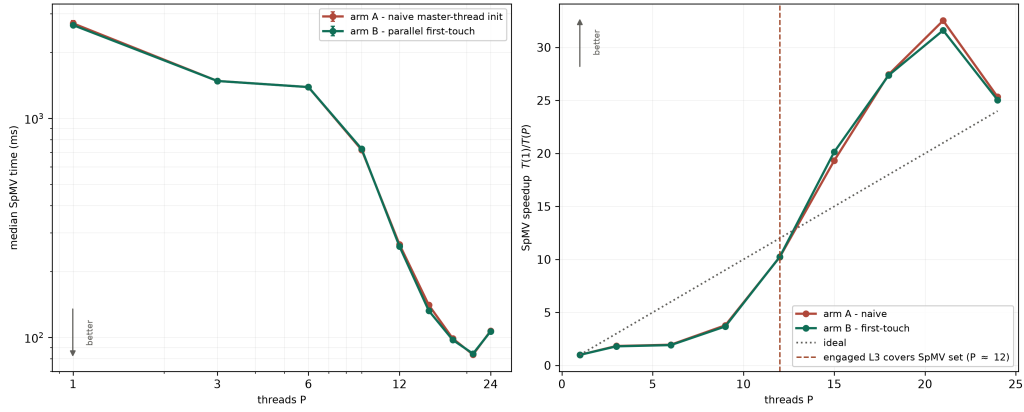


Figure A.8.: The initialization-arm A/B contrast at $n = 61$ (Section 7.3). Naive master-thread initialization and parallel first-touch are indistinguishable, which is the control that makes the schedule experiment necessary: on a single-NUMA socket with a per-CCX victim cache, residency is set by which core *accesses* a row, not by which thread first-touched the page.

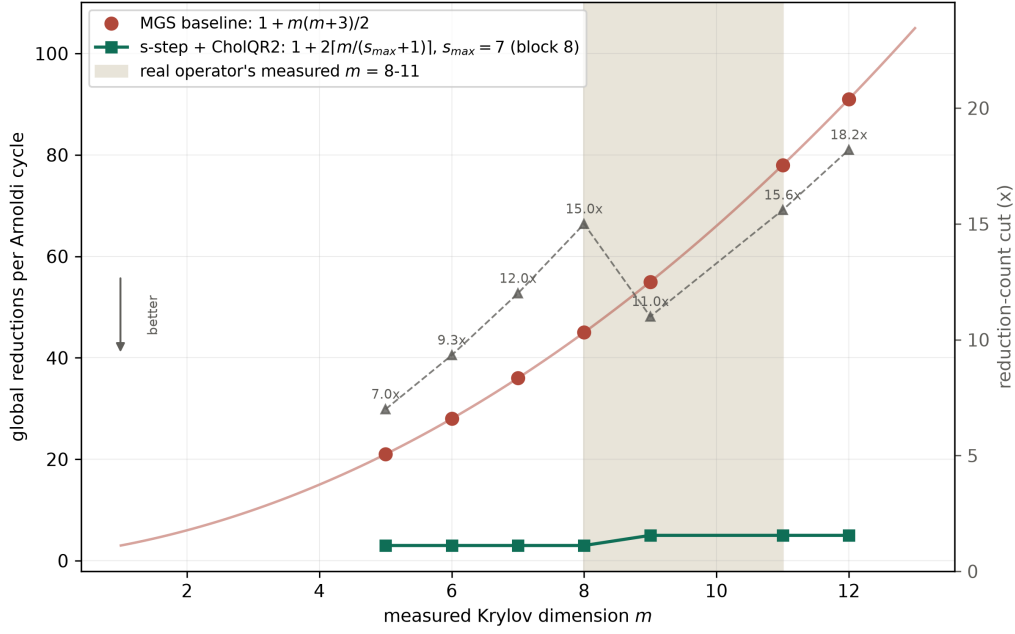


Figure A.9.: The reduction ledger of Table 8 drawn against the Krylov dimension: modified Gram–Schmidt’s $1 + m(m + 3)/2$ global reductions per Arnoldi cycle against the s -step arm’s $1 + 2\lceil m/(s_{\max} + 1) \rceil$. The cut is a count, not a time; Section 8.8 measures what it is worth.

B. SUPPLEMENTARY TABLES

Table B.1.: Weak scaling at a fixed local volume of $\approx 227,000$ rows per GPU. Cycle times are seven-repeat medians in ms/step; efficiency uses an arm-matched one-GPU run at $n = 61$ as its denominator. The $n = 97$ Basket arm is the predeclared convergence stop of Section 10.7 and carries no efficiency.

Arm	Configuration	$s=1$	Eff.	$s=4$	Eff.	CA
as-measured	Two V100s, $n=77$, Basket	6.883	43.1%	6.948	41.1%	0.99×
as-measured	Two V100s, $n=77$, Rainbow	6.876	43.1%	6.946	40.9%	0.99×
as-measured	Four V100s, $n=97$, Basket	stopped: convergence gate				
as-measured	Four V100s, $n=97$, Rainbow	8.871	33.4%	7.268	39.1%	1.22×
exact-depth	Two V100s, $n=77$, Basket	5.458	44.3%	5.274	50.7%	1.03×
exact-depth	Two V100s, $n=77$, Rainbow	5.444	44.1%	5.367	49.7%	1.01×
exact-depth	Four V100s, $n=97$, Basket	stopped: convergence gate				
exact-depth	Four V100s, $n=97$, Rainbow	7.602	31.6%	5.754	46.3%	1.32×

Table B.2.: Halo-exchange latency from the participant calibration, on idle V100s across two nodes. Seven-repeat medians; payloads are the exchanges the solver performs at each grid and block width. Adding participants taxes the all-reduce (Table 22) far more than it taxes the halo, which is a point-to-point exchange with a fixed neighbor.

Exchange	Payload	2 participants	4 participants	Ratio
$n=61, s=1$	29.1 KiB	36.80 μ s	39.05 μ s	1.06 $\times$
$n=61, s=4$	116.3 KiB	64.00 μ s	68.58 μ s	1.07 $\times$
$n=77, s=1$	46.3 KiB	46.70 μ s	49.43 μ s	1.06 $\times$
$n=77, s=4$	185.3 KiB	71.37 μ s	81.96 μ s	1.15 $\times$
$n=97, s=1$	73.5 KiB	55.11 μ s	59.57 μ s	1.08 $\times$
$n=97, s=4$	294.0 KiB	94.26 μ s	112.69 μ s	1.20 $\times$

Table B.3.: The allocation model: largest odd grid admitted per topology and block width, from the device’s reported free memory with a 10% reserve, evaluated before any allocation was made. The selected common grid is the minimum over all sixteen arms, $n = 381$ (55,306,341 rows), whose one-V100 $s=4$ allocation was predicted to sit 173 MiB below the reserve-limited capacity—and did allocate.

Topology	Basket		Rainbow	
	$s=1$	$s=4$	$s=1$	$s=4$
One V100	393	381	393	381
Two V100s, one node	495	481	495	481
Two V100s, two nodes	495	481	495	481
Four V100s, two nodes	623	605	625	605

Table B.4.: The independent referee’s canonical reference set, generated on an idle V100 with the fixed Al-Mohy–Higham Taylor action [2]. The Basket payoff’s boundary forcing inflates the augmented one-norm, so the fixed contract selects an impractical substep count and a scaled variant is used instead; Rainbow has no forcing tail and keeps the fixed contract.

Payoff	n	Rows	Substeps	SpMV	Operator rel. error
Basket	77	456,536	135	2,760	5.80×10^{-16}
Rainbow	77	456,533	142	2,739	7.34×10^{-16}
Basket	97	912,676	215	4,363	6.17×10^{-16}
Rainbow	97	912,673	226	4,351	6.34×10^{-16}

Table B.5.: Per-step communication account for the corrected arm at $n = 61$ on four V100s across two nodes, as the solver reports it. Collectives and halo exchanges are counted from the run; the host-synchronization stall is measured as the maximum over ranks and is an upper bound on recoverable overhead, because the host is waiting for work the device is already performing.

Quantity (per step)	$s=1$	$s=4$
CA blocks	8.78	2.41
Collectives	38.12	12.64
of which cross-rank agreement	4.00	4.00
Collective payload	3,325 B	2,945 B
Halo exchanges	8.78	4.82
Halo volume	1,531 KiB	1,681 KiB
Host synchronizations	35.12	10.64
measured stall	2.038 ms	1.639 ms
Calibrated collective floor	0.842 ms	0.279 ms
Calibrated halo-bandwidth floor	0.215 ms	0.236 ms
Cycle time	4.398 ms	2.401 ms